

INDIAN INSTITUTE OF TECHNOLOGY BOMBAY

Final Report

Summer of Science 2026

Spaceship Titanic Prediction

CS03 – Artificial Intelligence and Machine Learning

**Name: Mohit Khyalia
24B2289**

Mentor: Addepalli Bhargava Sarma

July 2026



Abstract

This report covers the full eight weeks of my Summer of Science 2026 project under CS03 (Artificial Intelligence and Machine Learning), ending in a binary classification model for the Spaceship Titanic dataset from Kaggle. The first four weeks were spent building up Python and the data science stack, the mathematics behind vectors, matrices, loss functions and gradient descent, data preprocessing, exploratory data analysis, and the theory and from-scratch implementation of Linear and Logistic Regression, alongside six applied mini-projects. Support Vector Machines, cross-validation, GridSearchCV, Decision Trees, Ensembles, and Deep Learning were either in progress or not yet reached by the halfway point, and I was upfront about those gaps rather than pretending otherwise.

The second four weeks closed those gaps and delivered the actual project. Week 5 finished Decision Trees, ensembles, and regularisation, then carried out a full exploratory analysis of the Spaceship Titanic dataset that ended in a concrete feature engineering plan — the CryoSleep × spending interaction turned out to be the strongest signal in the data, and shows up again at almost every later stage of this report. Week 6 built neural network foundations and a leakage-safe preprocessing pipeline that gets reused unmodified through every notebook after it. Week 7 used that pipeline to build the strongest classical model achievable — a six-model comparison under identical cross-validation, GridSearchCV tuning, feature importance validation against the EDA, threshold optimisation, and a full error analysis — ending in a tuned Gradient Boosting classifier. Week 8 applied a Keras neural network to the identical preprocessed data, ran a systematic architecture and regularisation search, and compared the result against the Week 7 model on the same validation fold under the same metrics. Gradient Boosting came out ahead (validation accuracy 0.8106 versus 0.7991 for the best neural network) and is the model behind the final Kaggle submission.

The tone stays consistent throughout this is a working account of the project as it actually happened, written with the same honesty whether describing something that worked cleanly or something that didn't. Several results along the way did not come out the way I expected: a data-leakage demonstration on one dataset showed no measurable difference where one was predicted, a Gradient Boosting model ranked best on a single train/test split but worst under cross-validation, and a final neural network configuration underperformed several of its own component experiments. I've tried to explain these results honestly rather than smooth them into a cleaner story than the data actually supports.

Table of Contents

Abstract.....	2
Table of Contents.....	3
1. Introduction	5
1.1 Overview of the Summer of Science Project.....	5
1.2 Artificial Intelligence, Machine Learning, and Deep Learning	5
1.3 Types of Machine Learning	5
1.4 Tools and Technologies Used	6
2. Mathematical Foundations	6
2.1 Vectors and Vector Operations.....	6
2.2 Matrices and Matrix Operations	7
2.3 Loss Functions and Gradient Descent	7
2.4 A Brief Note on Statistics and Probability.....	8
3. Data Preprocessing.....	9
3.1 Handling Missing Values	9
3.2 Data Standardization and Scaling.....	10
3.3 Encoding Categorical Variables.....	10
3.4 Train-Test Split	10
3.5 Handling Imbalanced Datasets	11
4. Exploratory Data Analysis	11
4.1 Summary Statistics.....	11
4.2 Visualization Techniques	11
4.3 Correlation Analysis.....	12
5. Linear Regression.....	13
5.1 Mathematical Formulation	13
5.2 Gradient Descent for Linear Regression	14
5.3 Implementation Study	14
6. Logistic Regression.....	15
6.1 Sigmoid Function and Mathematical Formulation	15
6.2 Log-Loss and Gradient Descent	16
6.3 Implementation Study	16
7. Support Vector Machines — An Introductory Overview	17
7.1 Margin and Support Vectors	17
7.2 Current Progress	18
8. Model Evaluation and Validation Concepts	18
8.1 Overfitting, Underfitting, and Bias-Variance Tradeoff	18

8.2 Cross-Validation and Hyperparameter Tuning	19
8.3 Evaluation Metrics	20
9. Applied Mini-Projects	21
9.1 Heart Disease Prediction	21
9.2 House Price Prediction	21
9.3 Loan Status Prediction.....	22
10. Advanced Classical ML (Week 5).....	24
10.1 Closing the Remaining Classical ML Gaps	24
10.2 Spaceship Titanic: Exploratory Data Analysis	25
10.3 Feature Engineering Decisions	28
10.4 Pipelines and the Leakage Question	28
11. Building Reusable Infrastructure (Week 6)	29
11.1 Neural Network Foundations	29
11.2 Choosing a Training Configuration Before Scaling Up	29
11.3 The Final Preprocessing Pipeline	30
12. Classical ML Final Model (Week 7)	31
12.1 Baseline Model Comparison	31
12.2 Hyperparameter Tuning	32
12.3 Feature Importance Validation.....	33
12.4 Threshold Optimisation and Error Analysis	33
12.5 Final Classical Submission	35
13. Deep Learning Final Model (Week 8).....	36
13.1 Reapplying the Week 6 Configuration at Scale	36
13.2 Architecture Search	36
13.3 BatchNormalization and Regularisation	37
13.4 Optimiser and Learning-Rate Scheduling	37
13.5 Final Model Assembly and a Result I Didn't Expect	38
13.6 Classical vs Deep Learning: Head-to-Head	38
14. Results and Comparative Analysis	40
16. Conclusion	42
References	42

1. Introduction

1.1 Overview of the Summer of Science Project

Summer of Science is a mentorship program at IIT Bombay where you spend the break going deep into a topic of your choice with a senior mentor guiding you. I picked CS03, the AI/ML track, mostly because I'd been using tools like ChatGPT a lot without having any real idea what was happening underneath. The end goal, as described in the Project Brief, is to build a model that predicts whether passengers on the fictional Spaceship Titanic were transported to an alternate dimension, based on things like their home planet, cryosleep status, cabin, age, and how much they spent on different onboard services. It's a binary classification problem taken from an actual Kaggle competition.

The project follows a Plan of Action spread across eight weeks, building up the mathematical and classical machine learning foundations before applying them to a single real Kaggle competition: the Spaceship Titanic dataset. The chapters that follow walk through that progression in the order it actually happened — Python, mathematics, preprocessing, and classical modelling first, learned in that order and practised on smaller datasets — so that by the time the report reaches the Spaceship Titanic work itself in Chapter 10, none of the underlying tools are new.

Before starting, I genuinely didn't know the difference between a parameter and a hyperparameter, or why a model even needs to be “trained” instead of just being told the rules. By the end of this foundational stretch, I wouldn't say I'd mastered any of it, but these ideas made sense to me in a way they hadn't before, and I could follow what was happening in a Logistic Regression implementation step by step instead of just running cells and hoping for the best.

1.2 Artificial Intelligence, Machine Learning, and Deep Learning

These three terms get thrown around together so often that I had to sit down and actually figure out how they relate. From my understanding, AI is the broadest umbrella basically any approach that makes a machine act intelligently, regardless of how it's done. ML is a subset of that where, instead of hand-coding rules, you give the computer examples and let it figure out the pattern on its own. DL goes a level deeper and uses neural networks with many layers, and tends to be the go-to for things like image recognition and language tasks.

Since this project works with structured tabular data just rows and columns, like a spreadsheet classical ML methods like Linear Regression, Logistic Regression, SVMs, and Decision Trees make more sense as a starting point than jumping straight to Deep Learning. That's part of why the first few weeks stick to these methods instead of neural networks.

1.3 Types of Machine Learning

Machine Learning is usually split into three broad categories. Supervised Learning is when the dataset already has the correct answers (the “labels”) attached, and the model learns to predict that label for new data it hasn't seen. This splits further into regression (predicting a number, like a house price) and

classification (predicting a category, like whether a loan got approved). Unsupervised Learning is when there are no labels at all, and the model just tries to find some structure on its own grouping similar points together, for example. Reinforcement Learning is a different setup entirely, where an agent learns through trial and error by getting rewarded or penalised for its actions.

The Spaceship Titanic problem falls under Supervised Learning, specifically binary classification, since “Transported” is just True or False. This is part of why Logistic Regression, which I get into in Chapter 6, has felt like one of the more directly useful things I've picked up so far.

1.4 Tools and Technologies Used

Week 1 was almost entirely Python basics and the libraries used throughout the rest of the course data types, lists, tuples, sets, dictionaries, loops, functions, that kind of thing. None of it was completely new since I'd done some Python before, but it was a useful refresher, especially the bits around list comprehensions and dictionary methods, which I'd honestly half-forgotten.

After that came NumPy, Pandas, Matplotlib, and Seaborn. I'll be honest Pandas syntax confused me at first, particularly the difference between `.loc[]` and `.iloc[]` for indexing, and I had to go back over that more than once before it actually clicked. NumPy felt a bit more natural since it's mostly array operations, but I didn't really get why everyone insists on NumPy arrays over plain Python lists until I started doing vector and matrix work in Week 3 (covered in Chapter 2).

Google Colab has been my main coding environment throughout, mostly because it doesn't need any local setup. For a few of the mini-projects, datasets were pulled in directly using the Kaggle API from inside the notebook, which was new to me I hadn't realised you could authenticate with Kaggle and grab a dataset in basically one line.

2. Mathematical Foundations

2.1 Vectors and Vector Operations

Week 3 is where the math starts, and the first piece of it is Linear Algebra vectors specifically. A vector, in this context, is just an ordered list of numbers. A data point with three features (say age, income, and a spending score) can be written as a three-component vector. The vector exercises walked through addition, subtraction, scalar multiplication, the dot product, and the magnitude (or norm) of a vector.

The dot product is the sum of the products of corresponding components of two vectors:

$$\mathbf{a} \cdot \mathbf{b} = a_1b_1 + a_2b_2 + \dots + a_nb_n$$

While trying this out with `np.dot(a, b)`, something that stuck with me was just how much faster NumPy is than writing the same thing as a loop the difference becomes obvious once the arrays get larger, and seeing that timing gap firsthand made the whole “vectorisation” idea click in a way reading about it never quite did.

Vector Dot Product Illustration

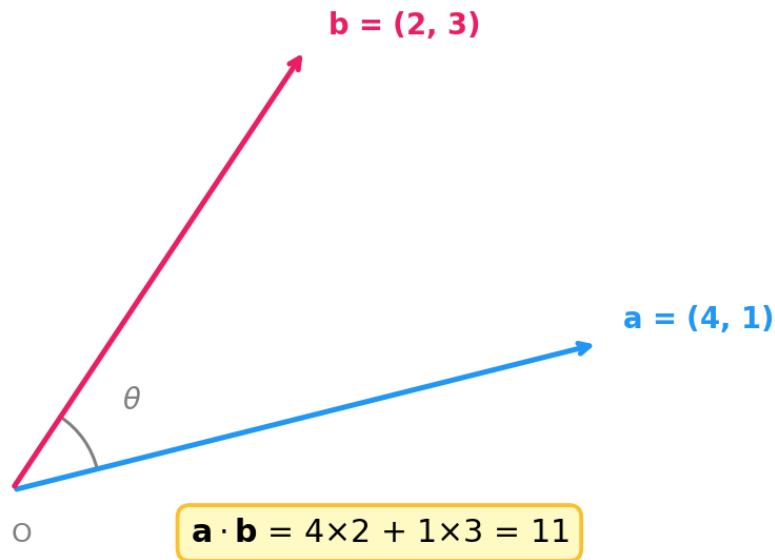


Figure 1: A geometric illustration of two vectors and their dot product, showing the angle θ between them and the projection of $\mathbf{b}$ onto $\mathbf{a}$.

2.2 Matrices and Matrix Operations

After vectors came matrices basically 2D arrays of numbers, where a dataset itself can be thought of as a matrix with rows as data points and columns as features. This covered addition, multiplication, transpose, and briefly the inverse and determinant of a square matrix.

One thing that helped tie everything together was realising matrix multiplication is really just a generalisation of the dot product each entry in the result is the dot product of a row from the first matrix and a column from the second. Once I saw that connection between Sections 2.1 and 2.2, the two topics stopped feeling like separate rules to memorise.

I'll admit the matrix inverse and determinant felt a bit abstract since gradient descent which is what I'm using for both Linear and Logistic Regression doesn't actually need a matrix inverse. From what I understand, the inverse becomes relevant for the Normal Equation approach to Linear Regression, which is an alternative to gradient descent, but I hadn't looked into that properly.

2.3 Loss Functions and Gradient Descent

This is probably the most important part of this chapter, since it sets up everything in Chapters 5 and 6. A loss function (or cost function) measures how wrong a model's predictions are compared to the actual values. For regression, the usual choice is Mean Squared Error:

$$J(w, b) = (1/m) \sum_i (\hat{y}_i - y_i)^2$$

where $\hat{y}_i$ is the predicted value, y_i is the actual value, and m is the number of training examples. Training a model basically means finding the w and b that make this cost as small as possible.

Gradient Descent is how you actually get there start with some random w and b , then keep nudging them in the direction that reduces the cost, until it stops improving by much:

$$w := w - \alpha (\partial J / \partial w)$$

$$b := b - \alpha (\partial J / \partial b)$$

α here is the learning rate how big a step you take each time. This is also where the difference between parameters and hyperparameters finally made sense to me. w and b are parameters that the model learns on its own during training. The learning rate and number of iterations are hyperparameters I choose those myself before training even starts, and they affect how training behaves rather than what's actually being predicted.

What surprised me was realising the exact same gradient descent process is used for both Linear Regression (Chapter 5) and Logistic Regression (Chapter 6) only the cost function and the formula for $\hat{y}$ change. I used to think of these as two completely separate algorithms, but once I saw the gradient descent loop written out for both, they started feeling like two versions of the same underlying idea.

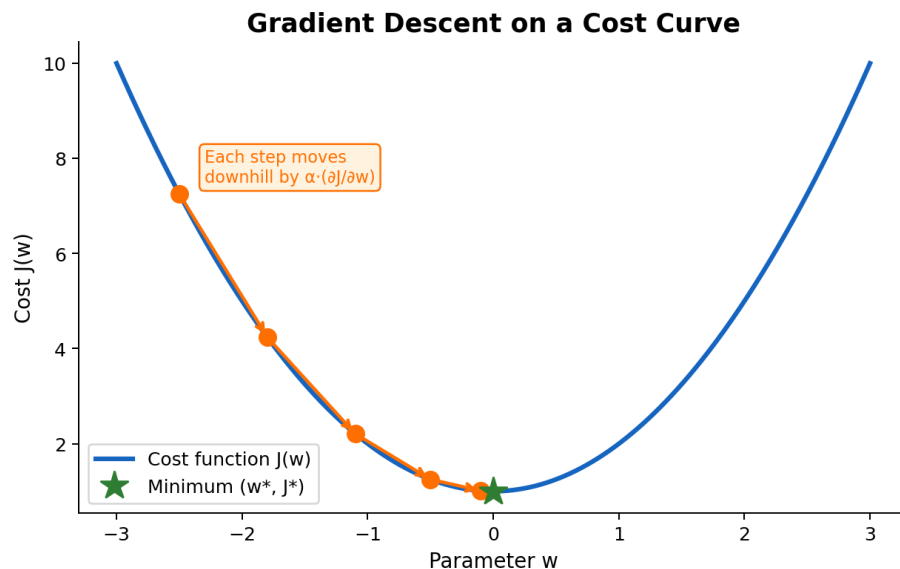


Figure 2: A simplified illustration of gradient descent a curve representing the cost function, with steps showing how the parameter value moves toward the minimum. The actual convergence plot from my own implementation appears later as Figure 7.

2.4 A Brief Note on Statistics and Probability

The Plan of Action lumps Statistics and Probability together with Linear Algebra under Week 3's "Math for ML" heading, and I want to be upfront that this is the weakest part of my Week 3 progress so far. I've picked up the basics mean, variance, standard deviation mostly through the preprocessing work, where

the σ in the StandardScaler formula (Section 3.2) is literally just the standard deviation. Probability distributions and conditional probability were still pretty shaky for me, and I knew they'd likely come up again, possibly with things like Naive Bayes. I'm flagging this honestly here rather than pretending I'd covered it properly, because I hadn't.

With that out of the way, the next two chapters move into Data Preprocessing and EDA, which is technically Week 2 material rather than Week 3 I put the math chapters first because Chapters 5 and 6 depend on them directly, but preprocessing and EDA are honestly where I feel most comfortable out of everything covered so far.

3. Data Preprocessing

Week 2 was all about Data Preprocessing and EDA, and looking back, this is probably the part I'm most confident about it got the most dedicated practice time, and I ended up reusing almost every technique here again in the Heart Disease, House Price, and Loan Status projects later on. Given that the Project Brief mentions the Spaceship Titanic dataset has missing values and a mix of categorical and numeric columns, I expected this to be the part of Weeks 1–4 that would matter most once the real project work began.

3.1 Handling Missing Values

Real datasets almost never come clean some rows just won't have a value for a given column. There are basically two ways to deal with this: drop the rows or columns using `dropna()`, or fill in the gaps using `fillna()` with something like the column's mean, median, or mode depending on whether it's numeric or categorical.

Dropping rows is the easy option, but it can throw away a lot of data if the missing values are spread out, which is why imputation tends to be the better call unless the missing chunk is tiny. This came up directly in the Loan Status Prediction project later a few columns each had a handful of missing entries, and I filled the numeric ones with the median and the categorical ones with the mode instead of just dropping rows and losing data for no good reason.

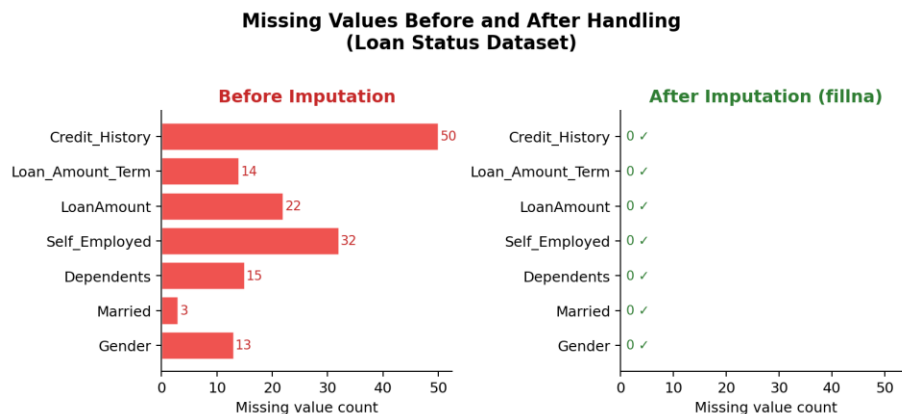


Figure 3: Comparison of the number of missing values in each column of the dataset before and after applying `fillna()`, showing the columns reduced to zero missing values. Source: Output from the missing value handling exercise.

3.2 Data Standardization and Scaling

The scikit-learn `StandardScaler` rescales each feature so it has a mean of 0 and a standard deviation of 1:

$$x_scaled = (x - \mu) / \sigma$$

where μ is the mean and σ is the standard deviation of that feature. This became a lot more real to me once I saw why it actually matters in the Diabetes Prediction project, features like “Glucose” (values in the hundreds) and “BMI” (values in the twenties or thirties) are on completely different scales. For something like SVM, which relies on distances between points, a feature with naturally larger numbers would end up dominating the distance calculation just because of its scale, not because it's actually more important. Scaling fixes that before training even starts.

3.3 Encoding Categorical Variables

Most ML models only understand numbers they can't directly work with text categories like “Male”/“Female” or “Earth”/“Mars”/“Europa”. `LabelEncoder` handles this by turning each unique category into an integer (0, 1, 2, ...), and there's also one-hot encoding via `pd.get_dummies()`, which instead creates a separate binary column per category.

I hadn't had to think too hard about which to use, since the mini-projects up to then mostly went with Label Encoding for simplicity. But from what I understand, it can be a bit misleading for categories that don't have any real order like planet names because the model might end up treating “2” as somehow bigger than “1” when really they're just different labels. Something to keep in mind for the Spaceship Titanic dataset later, since columns like `HomePlanet` and `Destination` don't have an obvious ordering, so one-hot encoding might actually be the better fit there though I hadn't tested this myself at the time.

In the Loan Status project (Section 9.3), there were six categorical columns, and I went with Label Encoding on all of them to match the approach I'd been following. Looking back at it now while writing this, `Property_Area` (Urban, Semiurban, Rural no real ordering) is probably exactly the kind of column where one-hot encoding would have made more sense. Something to revisit before doing the same thing on `HomePlanet` and `Destination` later.

3.4 Train-Test Split

`train_test_split()` splits a dataset into a training set (used to fit the model) and a test set (used to check performance on data it hasn't seen). The mini-projects mostly used an 80/20 split via the `test_size` parameter, with `random_state` fixed so the split is reproducible.

For classification problems where the classes aren't evenly split, the `stratify` parameter keeps the class proportions consistent between the train and test sets. I used this in a few of the mini-projects where the target wasn't perfectly balanced.

3.5 Handling Imbalanced Datasets

This is a problem I genuinely hadn't thought about before what happens if one class is way more common than the other? A model could hit 90% accuracy just by always predicting the majority class, without actually learning anything useful about the minority one. The fix is either resampling (oversampling the minority class or undersampling the majority) or using `class_weight` in scikit-learn to make the model pay more attention to the smaller class.

None of the mini-projects I've worked on so far have been severely imbalanced the Loan Status dataset (Section 9.3) leans a bit toward more approved loans than rejected ones, but not drastically. Still good to keep in mind for the Spaceship Titanic dataset, since at this point I had no idea how balanced the “Transported” column actually was that's something the later EDA would need to check directly rather than assume.

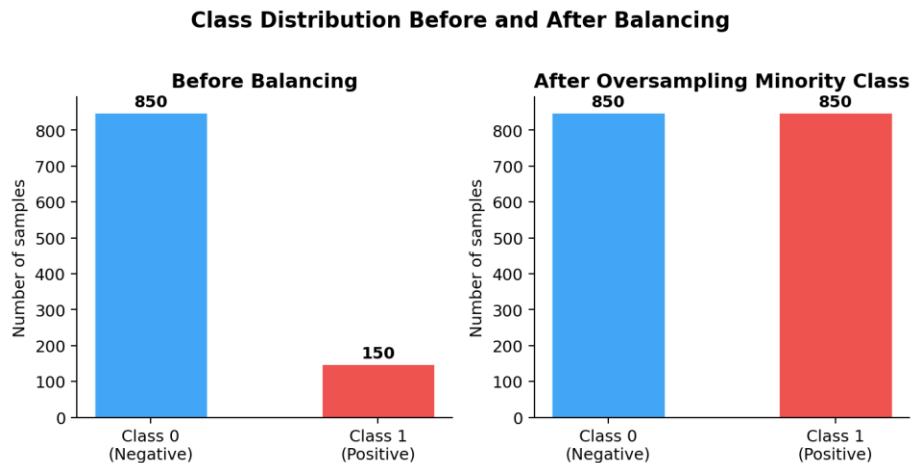


Figure 4: Bar chart comparing the number of samples in each class of the target variable before and after applying a balancing technique such as oversampling. Source: Output from the imbalanced dataset handling exercise.

4. Exploratory Data Analysis

EDA is basically getting to know your data before doing anything else with it looking at summary stats, plotting distributions, checking how features relate to each other. The Project Brief lists this as the first step for the Spaceship Titanic project, and it's also the first thing I did in every mini-project notebook.

4.1 Summary Statistics

`df.describe()` gives count, mean, std, min, max, and the 25th/50th/75th percentiles for every numeric column. I also lean on `df.info()` a lot it shows data types and non-null counts, so it tells you right away if there's missing data without needing a separate check.

4.2 Visualization Techniques

Matplotlib and Seaborn were the main tools here. Histograms are good for seeing the shape of a single numeric column whether it's roughly symmetric or skewed off to one side. Scatter plots show how two

numeric columns relate, and Seaborn's `pairplot()` gives you scatter plots for every pair of numeric columns at once, though I noticed it gets slow and cluttered fast once there are too many columns.

For categorical columns, `sns.countplot()` shows how many examples fall into each category, and combining it with the `hue` parameter let me check how loan approval varied across things like Education and Property_Area in the Loan Status project (Section 9.3).

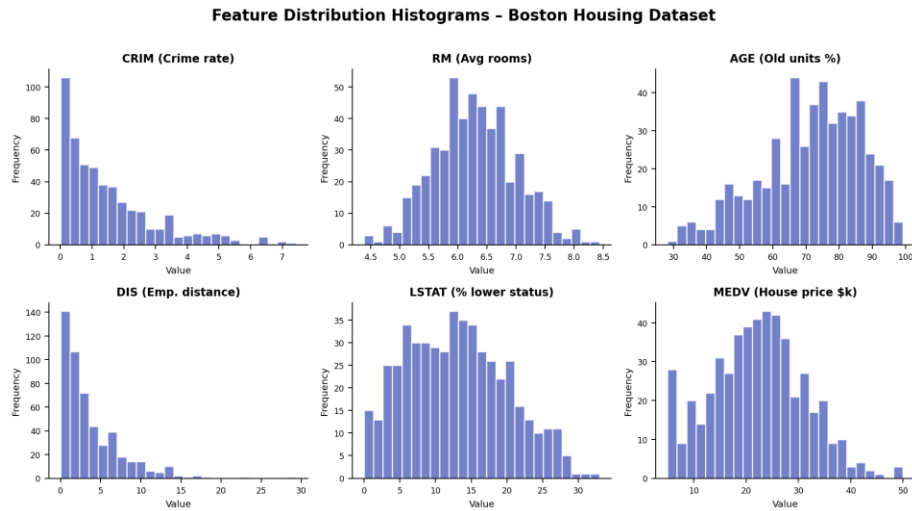


Figure 5: Histograms of selected numeric features from the House Price Prediction dataset, showing how each feature's values are distributed and whether they appear skewed. Source: Output from the House Price Prediction mini-project.

4.3 Correlation Analysis

Correlation measures how strongly two numeric columns move together, on a scale from -1 to 1. `df.corr()` computes this for every pair of numeric columns, and `sns.heatmap()` makes it way easier to spot strong relationships at a glance instead of squinting at a table of numbers.

In the Heart Disease project, I used this to get a rough first idea of which features seemed most tied to the presence of heart disease before training anything. One thing I had to keep reminding myself of a strong correlation between two features doesn't mean one causes the other, it just means they happen to move together in this particular dataset.

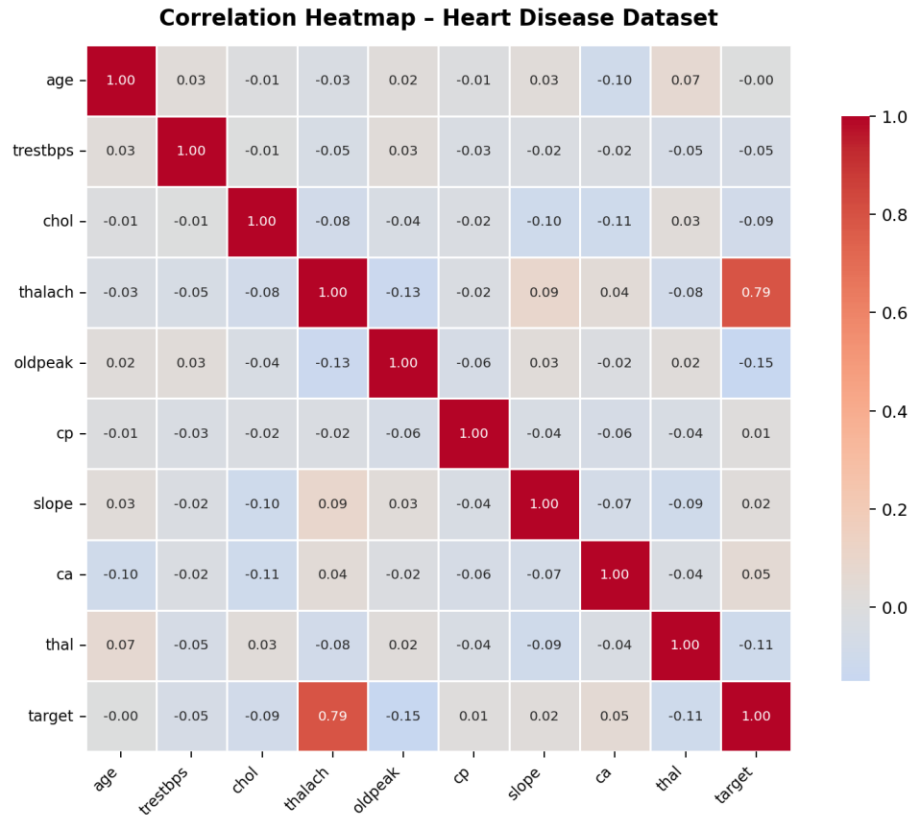


Figure 6: Heatmap showing the pairwise correlation between all numeric features and the target variable in the Heart Disease dataset, with darker/lighter shades indicating stronger positive or negative correlation. Source: Output from the Heart Disease Prediction mini-project.

5. Linear Regression

Linear Regression was the first model I actually built rather than just used not calling a library function, but writing out the gradient descent training loop myself. It's also the model applied in the House Price Prediction project (Section 9.2).

5.1 Mathematical Formulation

Linear Regression assumes the target y can be approximated as a linear combination of the input features x , plus a bias:

$$\hat{y} = wx + b$$

For one feature, w and b are just the slope and intercept of a line the picture I already had in my head from school. With multiple features, w becomes a vector of weights, one per feature:

$$\hat{y} = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

which connects back to the vector work from Chapter 2. The cost function is the same Mean Squared Error from Section 2.3:

$$J(w, b) = (1/m) \sum_i (\hat{y}_i - y_i)^2$$

5.2 Gradient Descent for Linear Regression

Minimising this cost needs its partial derivatives with respect to w and b :

$$\partial J / \partial w = (2/m) \sum_i (\hat{y}_i - y_i) x_i$$

$$\partial J / \partial b = (2/m) \sum_i (\hat{y}_i - y_i)$$

These plug into the gradient descent update rules from Section 2.3. Each iteration: compute predictions with the current w and b , measure how far off they are, work out the derivatives above, and nudge w and b a little in the direction that lowers the cost.

5.3 Implementation Study

I structured the from-scratch version as a small Python class with `__init__`, `fit`, and `predict` methods. `__init__` sets the learning rate and number of iterations, `fit` runs the gradient descent loop and stores the final w and b , and `predict` just applies $\hat{y} = wx + b$ to new inputs using those learned values.

While building this, I ran into a problem where the cost kept going up instead of down after a few iterations. After some debugging and rereading the explanation a couple of times I realised my learning rate was way too high, which made the updates overshoot the minimum instead of approaching it gradually. Once I lowered it, the cost curve actually flattened out the way it was supposed to, and honestly that was one of the more satisfying moments of the past four weeks.

I also compared the final w and b from my own implementation against scikit-learn's `LinearRegression` on the same data. They weren't identical, but close enough that I felt reasonably confident my version was doing the right thing rather than just spitting out plausible-looking numbers.

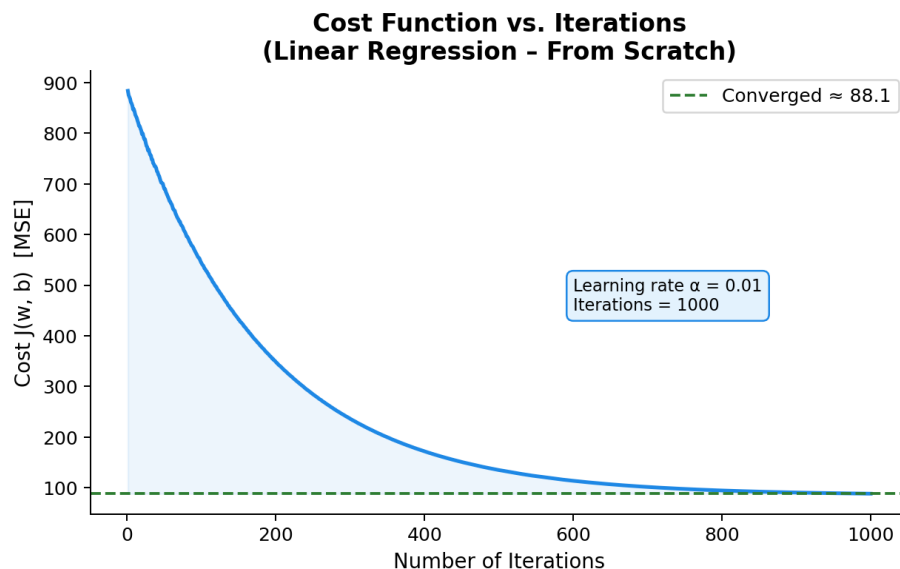


Figure 7: Plot of the cost function $J(w, b)$ against the number of gradient descent iterations during training of the from-scratch Linear Regression model, showing the cost decreasing and converging.

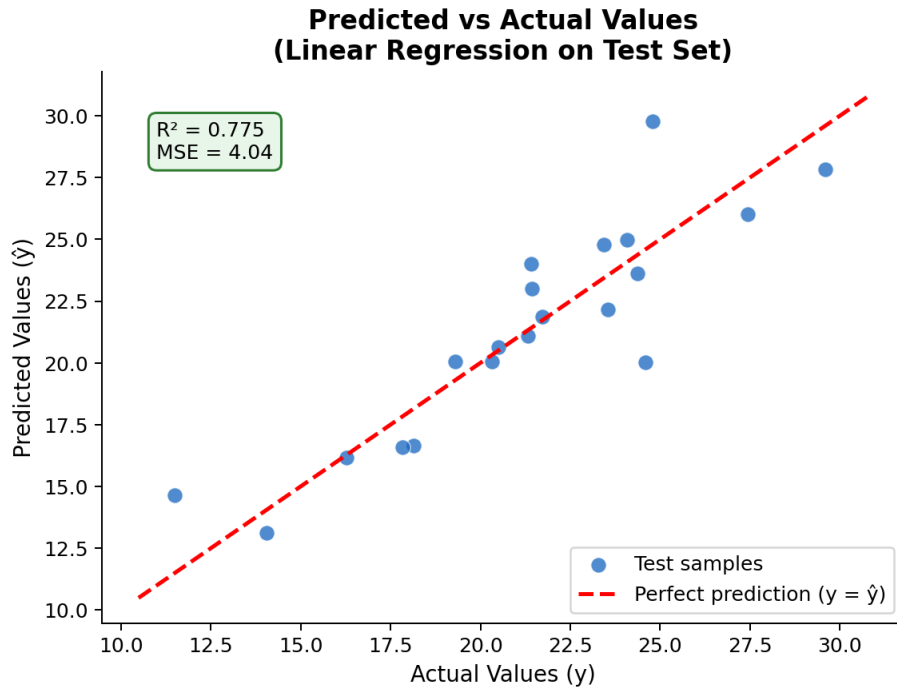


Figure 8: Scatter plot comparing the model's predicted values against the actual values on the test set; points closer to the 45-degree diagonal line indicate more accurate predictions.

6. Logistic Regression

If Linear Regression was the first model I built from scratch, Logistic Regression is probably the one that matters most for this specific project, since the Spaceship Titanic task is binary classification predicting True or False for “Transported” which is exactly what Logistic Regression is built for. I applied it to the heart disease dataset in Section 9.1.

6.1 Sigmoid Function and Mathematical Formulation

The issue with using Linear Regression directly for classification is that $\hat{y} = wx + b$ can output literally any number, but for classification you want something between 0 and 1 that behaves like a probability. Logistic Regression handles this by passing the linear combination through the sigmoid function:

$$z = wx + b$$

$$\hat{y} = \sigma(z) = 1 / (1 + e^{(-z)})$$

Sigmoid squashes any real number into the (0, 1) range big positive z gives an output near 1, big negative z gives something near 0, and $z = 0$ lands exactly on 0.5. A threshold of 0.5 then decides the final prediction: $\hat{y} \geq 0.5$ means class 1, otherwise class 0.

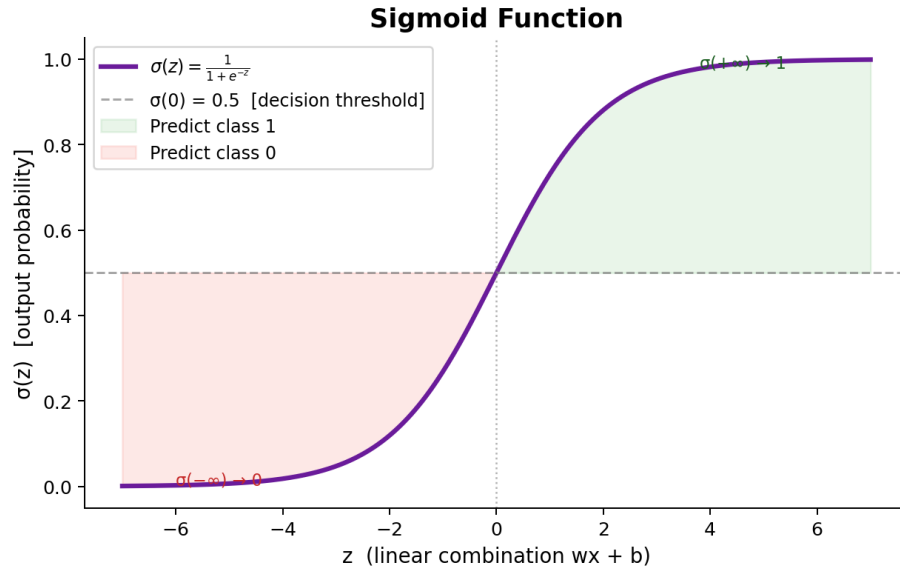


Figure 9: Plot of the sigmoid function $\sigma(z) = 1/(1+e^{-z})$ over a range of input values, showing its characteristic S-shape and how it maps any real number to the range (0, 1).

6.2 Log-Loss and Gradient Descent

MSE doesn't work well for classification combined with sigmoid it can produce a cost surface with multiple local minima, which messes with gradient descent. Logistic Regression uses log-loss instead:

$$J(w, b) = -(1/m) \sum_i [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

I'll be honest, this formula didn't make sense to me the first time I saw it. What helped was breaking it into cases: when the true label y_i is 1, the second term disappears and you're left with $-\log(\hat{y}_i)$, which is small when $\hat{y}_i$ is near 1 (correct) and blows up when $\hat{y}_i$ is near 0 (very wrong). The $y_i = 0$ case works the same way but mirrored. So the loss stays small when the prediction matches the label and grows fast when it doesn't.

The gradient descent rules end up looking almost the same as Linear Regression's (Section 5.2), just with $\hat{y}$ now coming from sigmoid instead of directly from $w x + b$:

$$\partial J / \partial w = (1/m) \sum_i (\hat{y}_i - y_i) x_i$$

$$\partial J / \partial b = (1/m) \sum_i (\hat{y}_i - y_i)$$

6.3 Implementation Study

Same class structure as Linear Regression `__init__`, `fit`, `predict` except `fit` now uses sigmoid and the log-loss gradients, and `predict` applies the 0.5 cutoff to turn probabilities into actual class labels.

This clicked further once I compared my version's accuracy against scikit-learn's `LogisticRegression` on the same Heart Disease train-test split close numbers, but not identical, which makes sense since sklearn likely uses a more refined optimiser than my basic gradient descent loop, plus some regularisation I hadn't added to mine.

Looking at the cases where the model got things wrong also made the Precision/Recall discussion in Section 8.3 feel a lot less abstract a few patients it misclassified were ones it predicted as “no heart disease” when they actually had it, which is precisely the kind of false negative Recall is meant to catch.

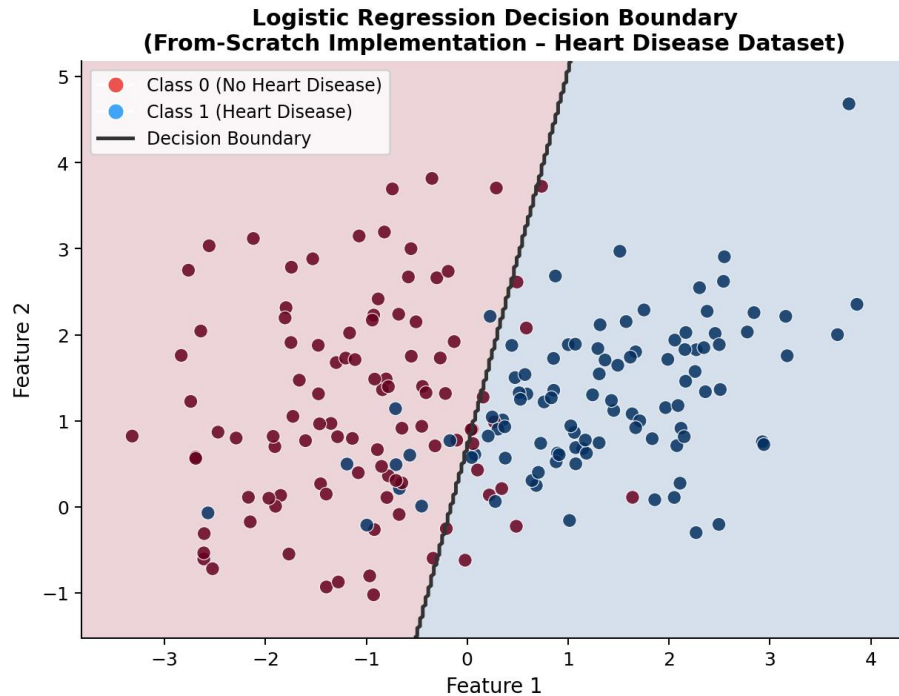


Figure 10: Decision boundary learned by the from-scratch Logistic Regression model, plotted over two of the dataset's features, showing the region classified as each class.

7. Support Vector Machines — An Introductory Overview

This was a topic I was still working through, so I want to be upfront that I understood it less deeply than Linear and Logistic Regression. SVMs and kernel methods are on the Week 4 plan, and I'd started the from-scratch material but hadn't finished working through it yet.

7.1 Margin and Support Vectors

From what I've got so far, an SVM tries to find a hyperplane (a line in 2D, a plane in 3D, and so on) that separates two classes while making the margin the gap between the hyperplane and the closest points from each class as wide as possible. The points sitting right on that margin, the ones effectively holding the boundary in place, are the support vectors. This is a different philosophy from Logistic Regression, which doesn't try to maximise any margin it just minimises log-loss across all the data.

Kernel methods come up too from what I can tell, they're a way to handle data that isn't separable by a straight line, by mapping it into a higher-dimensional space where it becomes separable. I'm leaving this part out of the explanation here since I've only half-read it and don't want to write about something I don't actually understand yet.

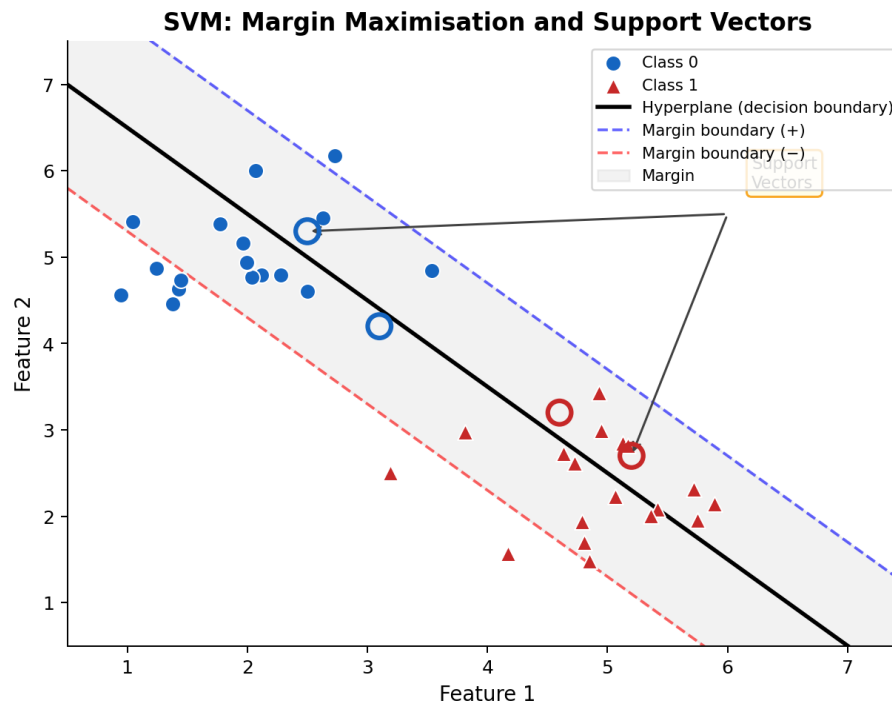


Figure 11: Diagram showing a separating hyperplane between two classes of points, the margin around it, and the support vectors (the points closest to the hyperplane from each class).

7.2 Current Progress

While I'm still working through the from scratch material, I did already use scikit-learn's readymade SVC in the Diabetes Prediction mini-project standardise the features first with StandardScaler, fit an SVC with a linear kernel, then check accuracy on both train and test sets. That gave me some practical feel for the sklearn side of things before tackling the math myself.

Over the rest of this week I'm planning to actually finish the from-scratch SVM material and properly understand hinge loss and how gradient descent applies to it, the same way I worked through Linear and Logistic Regression. I've deliberately left the hinge loss formula and any from-scratch code out of this report, since I don't think I understand it well enough yet to explain it properly without just repeating something I half-read.

8. Model Evaluation and Validation Concepts

The Project Brief wants models evaluated using Accuracy, Precision, Recall, and F1-score, plus cross-validation and hyperparameter tuning documented somewhere. These are also Week 4 topics, so my grasp here is still a work in progress but I've at least run into all of these ideas in some form across the mini-projects.

8.1 Overfitting, Underfitting, and Bias-Variance Tradeoff

Overfitting is when a model basically memorises the training data including all its noise to the point where it does great on training data but falls apart on anything new. Underfitting is the opposite: the model's too simple to even capture the real pattern, so it does badly on both training and test data. The Bias-Variance Tradeoff is the usual way of framing this high bias (too simple) tends toward underfitting, high variance (too sensitive to the training data) tends toward overfitting, and you're trying to land somewhere in between.

I've noticed hints of overfitting in a couple of the mini-projects, where training accuracy came out noticeably higher than test accuracy, but I hadn't properly studied the fixes for this yet regularisation being the obvious one, which gets covered properly in Section 10.1.

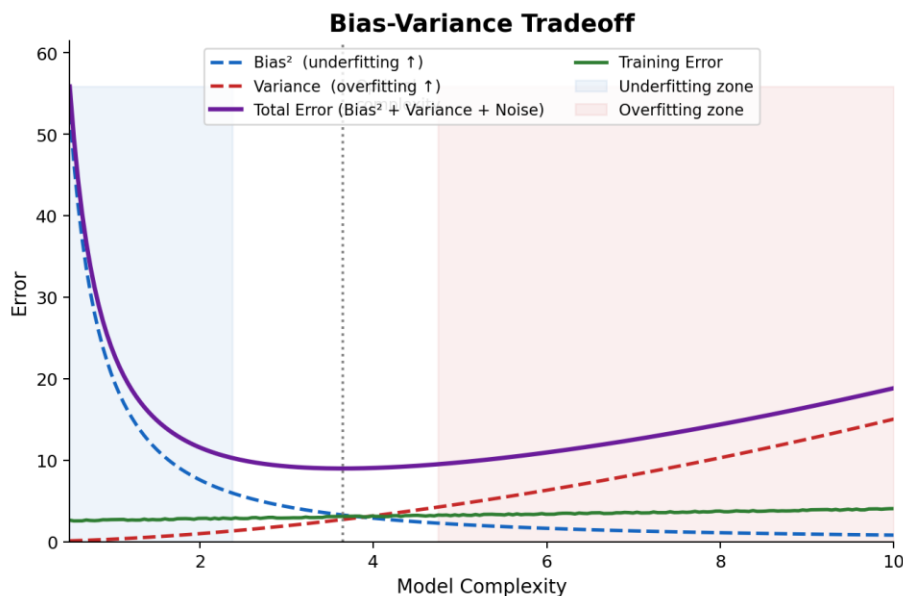


Figure 12: A typical plot showing training error and validation/test error as model complexity increases training error keeps decreasing, while validation error first decreases (underfitting region) and then increases again (overfitting region).

8.2 Cross-Validation and Hyperparameter Tuning

K-fold cross-validation splits the training data into k chunks and trains the model k separate times, each time holding out a different chunk as validation. Averaging the results across all k runs gives a more reliable sense of how the model generalises than just one train-test split would.

Hyperparameter tuning means searching for the best values of things like the learning rate, number of iterations, or regularisation strength, instead of just guessing. GridSearchCV in scikit-learn automates this by trying combinations of hyperparameters and scoring each with cross-validation.

I want to be honest that my understanding of both of these was mostly conceptual I'd read about them and understood why they're useful, but hadn't actually implemented k-fold cross-validation or a grid search in any of my own notebooks yet. Everything in Chapter 9 still uses a single train-test split. That gap closed properly once cross-validation and GridSearchCV actually got implemented, since the Project Brief explicitly wants hyperparameter tuning documented for the final project.

8.3 Evaluation Metrics

Accuracy the fraction of correct predictions is the simplest metric, but as I mentioned in Section 3.5, it can be misleading on imbalanced data. Precision and Recall give a sharper picture:

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

where TP, FP, and FN come from comparing predictions against actual labels. Precision is asking “of everything I predicted positive, how much was actually positive?”, while Recall asks “of everything that was actually positive, how much did I catch?”. F1 combines the two using their harmonic mean:

$$\text{F1} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

Precision, Recall, and F1 all come out of the confusion matrix, which is just a table comparing predicted labels against actual ones. Running `confusion_matrix()` on the Heart Disease Logistic Regression results gave me an actual TP/FP/FN/TN breakdown for the first time instead of a single accuracy number, and seeing the false negatives there is really what made Recall click for me those are the patients predicted as healthy who actually had heart disease, which in a real medical setting would obviously be the worse mistake to make.

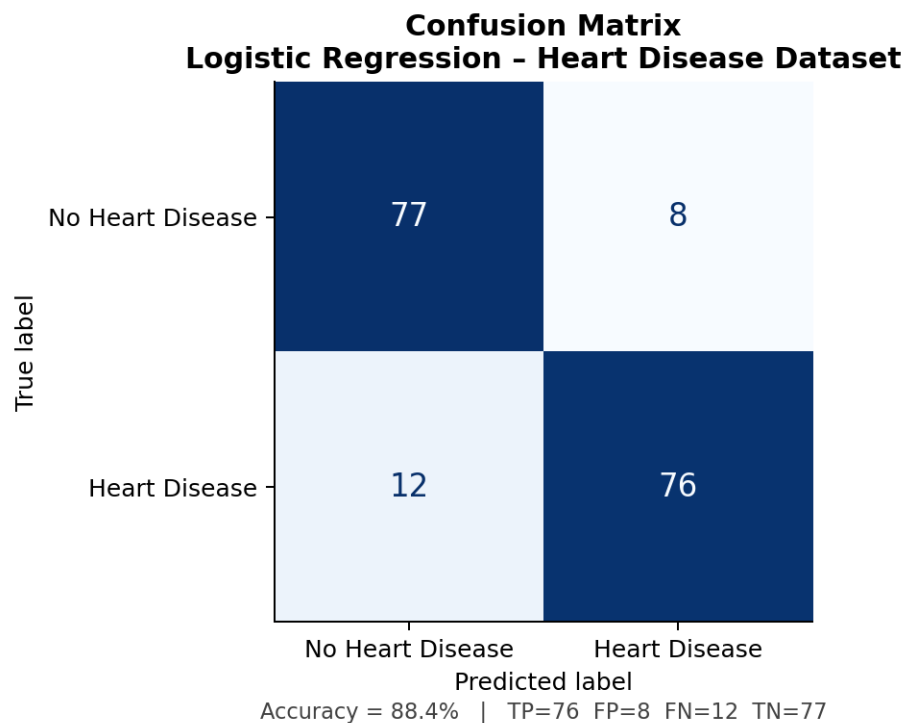


Figure 13: Confusion matrix showing the counts of true positives, true negatives, false positives, and false negatives for the Logistic Regression model applied to the Heart Disease dataset.

Up to that point, most of the mini-projects I'd finished only reported accuracy, since the classes were reasonably balanced and accuracy alone gave a fair picture. I'd seen Precision, Recall, and F1 show up in

scikit-learn's `classification_report` output, but hadn't yet made a habit of reporting all four metrics consistently something I wanted to start doing more deliberately going forward.

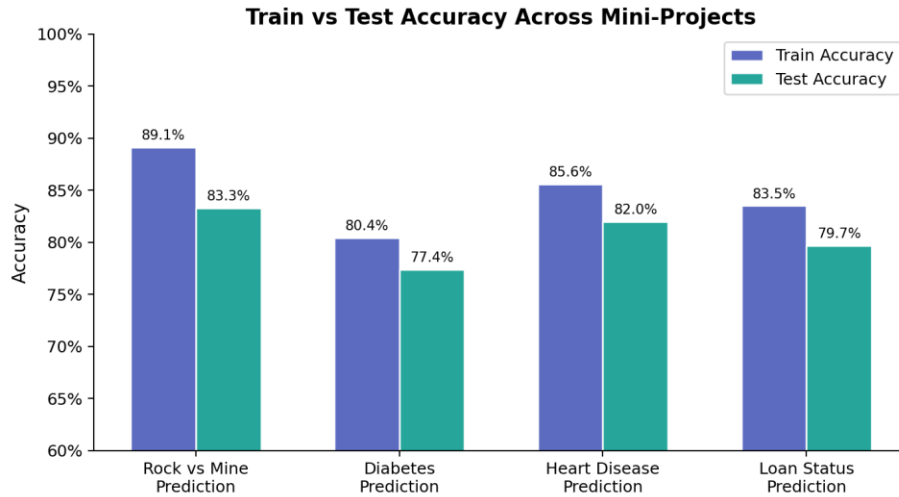


Figure 14: Bar chart comparing the test accuracy achieved across the Rock vs Mine, Diabetes, Heart Disease, and Loan Status Prediction mini-projects.

9. Applied Mini-Projects

This chapter goes into the three mini-projects that I think best show how preprocessing, EDA, and modelling actually come together on real data. Rock vs Mine, Diabetes, and Spam Mail Prediction get referenced earlier in the report in Sections 3.2, 7.2, and 8.3 as supporting evidence, but I haven't given them full write-ups here so this chapter doesn't get too long.

9.1 Heart Disease Prediction

This project uses a dataset where each row is a patient, with features like age, sex, resting blood pressure, cholesterol, and a handful of other clinical measurements, with the target column indicating whether the patient has heart disease. Since almost everything here is numeric, preprocessing was light mostly just checking for missing values (there weren't any) and applying `StandardScaler` before training.

EDA involved computing the correlation matrix and plotting it as a heatmap (Section 4.3), which gave me a first sense of which features looked most tied to heart disease chest pain type and max heart rate stood out with relatively higher correlation values. After splitting train/test, I trained a Logistic Regression model (Chapter 6) and checked accuracy the confusion matrix in Section 8.3 also comes from this same model. I picked this project first because the data was clean enough to let me focus entirely on the modelling side train/test split, Logistic Regression, and reading a confusion matrix properly rather than fighting messy inputs at the same time. That clean-data baseline is what the messier projects later in this chapter, and eventually the Spaceship Titanic pipeline, got compared against whenever preprocessing decisions got more complicated.

9.2 House Price Prediction

This one uses a housing dataset where the target is a continuous price, making it a regression problem, so I used Linear Regression (Chapter 5) instead of Logistic Regression here.

EDA mostly meant plotting histograms of the numeric features (Section 4.2) to see their distributions a couple of features looked noticeably skewed rather than evenly spread, which made me wonder whether some kind of transformation might help, since Linear Regression generally assumes a roughly linear relationship. I hadn't tested that idea at the time, though. After training, I evaluated using R^2 and Mean Squared Error, and plotted predicted prices against actual prices to see visually how close things were. This was also the first project where I worked with regression rather than classification, which made the distinction between the two evaluation mindsets R^2 and MSE instead of accuracy and a confusion matrix concrete rather than just something I'd read about. The skewed-feature observation here turned out to matter again much later: the same heavy right-skew showed up in the Spaceship Titanic spending columns, which is why a log transform was the obvious first thing to try there (Section 10.3).

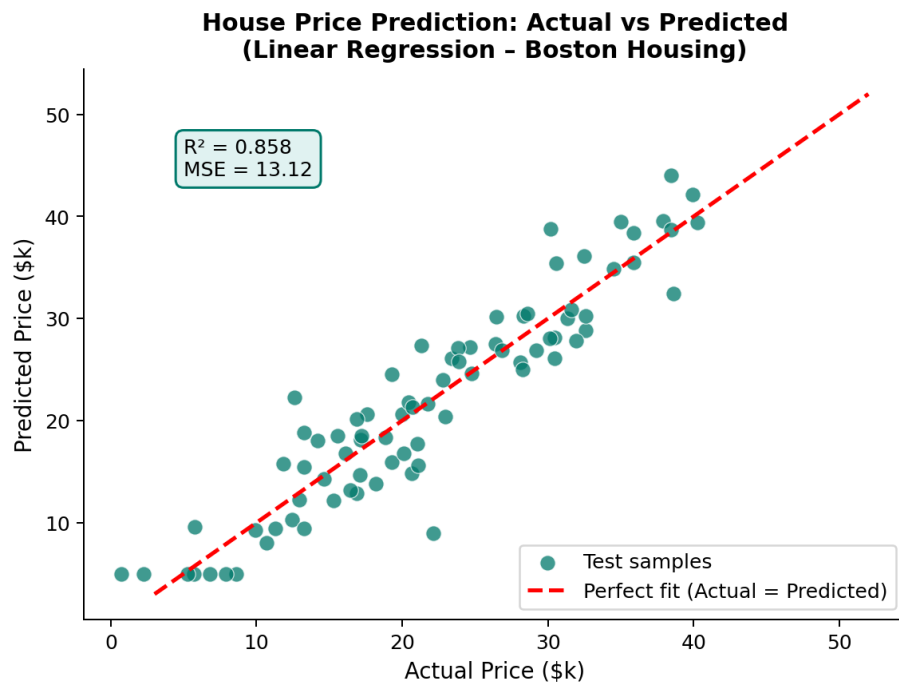


Figure 15: Scatter plot of actual house prices (x-axis) against the prices predicted by the Linear Regression model (y-axis) for the test set; points closer to the diagonal line represent more accurate predictions. Source: Output from the House Price Prediction mini-project.

9.3 Loan Status Prediction

Out of everything I've done so far, this one felt closest to the actual Spaceship Titanic work mainly because it was easily the messiest dataset of the three. The columns split fairly evenly between categorical and numeric: Gender, Married, Dependents, Education, Self_Employed, and Property_Area are categorical, while ApplicantIncome, CoapplicantIncome, LoanAmount, Loan_Amount_Term, and Credit_History are numeric (Credit_History is stored as 0/1 but really represents a yes/no whether the applicant has any credit history at all). The target, Loan_Status, is whether the loan got approved (Y) or not (N).

Running `data.info()` right at the start showed that Gender, Married, Dependents, Self_Employed, LoanAmount, Loan_Amount_Term, and Credit_History all had fewer non-null entries than the total row count, so handling missing values was the first real step. I filled the numeric columns (LoanAmount, Loan_Amount_Term) with their medians, and the categorical ones (Gender, Married, Dependents, Self_Employed, Credit_History) with their mode, since going with the most common value seemed reasonable given how few rows were actually affected.

After that I applied Label Encoding to all six categorical columns plus the target, used StandardScaler on the numeric columns, and did a stratified `train_test_split` since Loan_Status wasn't perfectly balanced there were noticeably more approved loans than rejected ones, which tied back to the imbalance discussion in Section 3.5, though I didn't end up resampling since the imbalance wasn't extreme. I trained an SVM classifier (the same setup from Section 7.2) and checked accuracy on the test set.

For EDA, alongside the usual `describe()` and correlation checks on the numeric side, I used `sns.countplot()` to see how Loan_Status broke down across categories like Education and Property_Area. Graduates, and applicants from semi-urban areas, both seemed to get approved at a noticeably higher rate than the other groups though with a dataset this small, I'm being cautious about reading too much into that.

What made this project stand out to me is that it forced me to combine basically everything from Chapters 3 and 4 in one go missing values, encoding, scaling, a stratified split, and EDA across both numeric and categorical features on a dataset that actually felt as messy as the Spaceship Titanic data would turn out to be, going by the Project Brief's description (which also mentions HomePlanet, Destination, and Cabin as categorical, and Age plus the spending columns as numeric, with missing values scattered across both types).

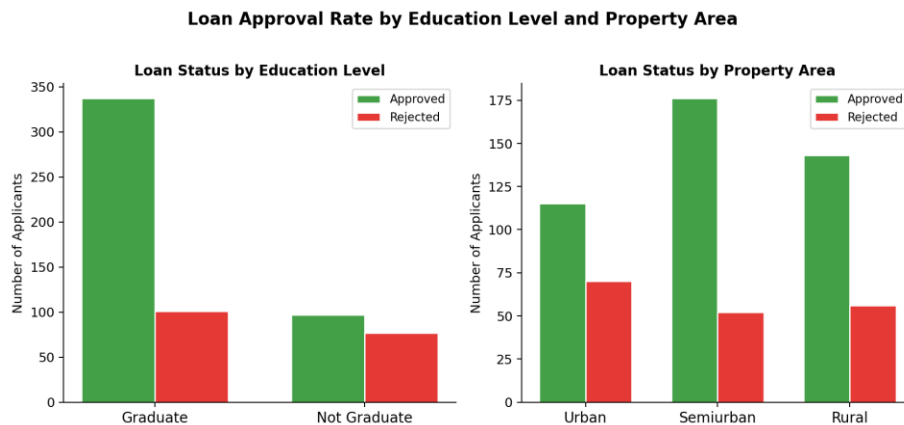


Figure 16: Count plots showing the number of approved versus rejected loans split by Education level and by Property Area, used to get a first look at how these categorical features relate to the target variable. Source: Output from the Loan Status Prediction mini-project.

10. Advanced Classical ML (Week 5)

The Spaceship Titanic dataset, from the Kaggle competition of the same name, is the actual subject of this project: a binary classification problem where each row is a passenger aboard a spacecraft, and the task is to predict whether that passenger was Transported to an alternate dimension during a collision with a spacetime anomaly. The columns include HomePlanet, CryoSleep, Cabin (which packs deck, number, and side of the ship into a single string), Destination, Age, VIP status, and spending across five onboard services — RoomService, FoodCourt, ShoppingMall, Spa, and VRDeck — alongside the Transported target itself.

Two things needed finishing before working with this dataset directly: Decision Trees, ensembles, and regularisation, which extend the Logistic Regression and SVM material from Chapters 6 and 7 into more flexible model families, and a proper exploratory pass over the Spaceship Titanic data itself. Both were done in that order — trees and ensembles first, on datasets already familiar from Chapter 9, so the underlying mechanics weren't new by the time they needed to be applied to Spaceship Titanic feature importances later in this chapter. The project's original scope had also included a first look at clustering and a Streamlit deployment; both were dropped once it became clear that a single supervised classification problem didn't need either, which freed up enough time to treat the Spaceship Titanic build as four unhurried stages — EDA and feature engineering, a reusable preprocessing pipeline, a tuned classical model, and a tuned neural network compared fairly against it — rather than a single rushed one.

10.1 Closing the Remaining Classical ML Gaps

I implemented and compared Decision Trees, Random Forest, and Gradient Boosting on the loan status dataset — the same one from the SVM mini-project in Section 9.3, which made it easy to see what an ensemble actually buys you on data I already had a feel for. I worked through Gini impurity and the tree-depth-vs-accuracy trade-off first on a single pruned tree (accuracy peaked around depth 4–6 and fell off after that, the same overfitting shape as the polynomial-degree experiments from Week 4), then compared Random Forest and Gradient Boosting against it under 5-fold cross-validation — the first time I'd actually run cross-validation myself rather than just reading about why it mattered.

decision_trees_and_ensembles.ipynb — 5-fold CV comparison, loan status dataset

	Model	CV Mean	CV Std
	Random Forest	0.808333	0.037616
	Decision Tree (depth=4)	0.791667	0.040074
	Gradient Boosting	0.777083	0.035232

This result actually surprised me a little, and I think it's worth being upfront about rather than smoothing over. On a single train/test split, Gradient Boosting looked like the best of the three (test accuracy 0.8438 against Random Forest's 0.8333), which is what I expected going in. But under 5-fold cross-validation, the ranking flips — Random Forest comes out ahead of both the single tree and Gradient Boosting, and Gradient Boosting is actually the weakest of the three on average, with the widest gap between its single-split result and its CV mean. My read on this is that Gradient Boosting, being the most flexible of the three at default settings, is also the most sensitive to which particular 80/20 split you happen to draw — a single split can flatter or punish it more than it does a more stable bagging method like Random Forest. It's a

small dataset (480 rows), so this kind of variance is more visible than it might be at scale. Whatever the exact reason, this is the result that convinced me a single train/test number isn't enough to trust, and it's the reason every comparison from here on — including the Spaceship Titanic model comparison in Section 12.1 — uses cross-validation rather than a single split.

Ridge and Lasso regularisation were covered separately on the house price dataset from Week 4, comparing coefficient shrinkage across a range of alpha values with the alpha itself chosen by cross-validation (RidgeCV picked 628.03, LassoCV picked 100.0) rather than by hand. The comparison on held-out test R^2 actually surprised me here too — I expected Lasso's feature selection to help generalisation more than it did. Ridge came out on top (test $R^2 = 0.849$), Lasso next (0.788, with 197 of the original 243 one-hot-expanded coefficients surviving to non-zero), and plain OLS clearly overfit (train $R^2 = 0.936$ but test R^2 only 0.683). What this told me is that most of the 245 features in this dataset carry at least a little genuine signal, which is exactly the situation Ridge's smooth, distribute-the-penalty-everywhere approach is suited to — Lasso's harder feature selection ends up throwing away features that weren't pure noise. I kept this in mind later when deciding not to be too aggressive about dropping features in the Spaceship Titanic pipeline in Section 10.3.

The last piece of Week 5 was `machine_learning_pipelines.ipynb`, which set out to demonstrate the data leakage problem Chapter 3 could only describe abstractly — every notebook through Week 4 fit preprocessing on the full dataset before splitting, which is technically unsafe even if it didn't cause visible problems on those particular datasets. I built a side-by-side comparison: manual preprocessing fit before the CV loop, versus a scikit-learn Pipeline with the preprocessing fit only inside each training fold. The result was a bit anticlimactic — both approaches gave exactly 0.6955 ± 0.0044 CV accuracy on the loan dataset, to four decimal places. On reflection this makes sense: the leak here is only in the scaling statistics for a handful of numeric columns, and `StandardScaler`'s mean and standard deviation don't shift much between a 4/5-fold training subset and the full dataset on data this size. But I don't think that makes the practice optional — it just means this particular dataset wasn't a case where the leak happened to matter. The risk scales with how much a preprocessing step can actually learn from data it shouldn't see (a high-cardinality target encoding would be a much worse case than a `StandardScaler`), and the only way to guarantee it never bites you is to default to the safe pattern every time, which is what Pipeline and ColumnTransformer became from this point on.

10.2 Spaceship Titanic: Exploratory Data Analysis

With the modelling toolkit actually complete, `spaceship_titanic_eda.ipynb` was my first real look at the competition dataset. The training set has 8,693 passenger records across 14 columns; the Kaggle test set has 4,277 records across 13 columns, missing the Transported label, since that's what has to be predicted and submitted.

The first thing I checked was missingness, since I'd been burned before by not checking it early enough. Every column had some missing data, but it was spread fairly evenly — roughly 2–2.5% missing per column, worst in `CryoSleep` at 2.50% and best in `Age` at 2.06%, with nothing that looked like a systematic pattern tying the missing rows together. That ruled out just dropping rows, which would have thrown

away a meaningful chunk of an already moderate-sized dataset, in favour of the median/most-frequent imputation from Week 2, applied per column.

The target variable, Transported, came out almost exactly balanced — a positive rate of 0.5036 in the full training set, confirmed directly from the preprocessing notebook's own output in Section 11.3. That was a genuinely useful early result, because it meant the class-imbalance techniques from Week 2 weren't needed here, and that plain accuracy would actually mean something as a metric for the rest of the project instead of needing the same caveats as the imbalanced mini-project examples.

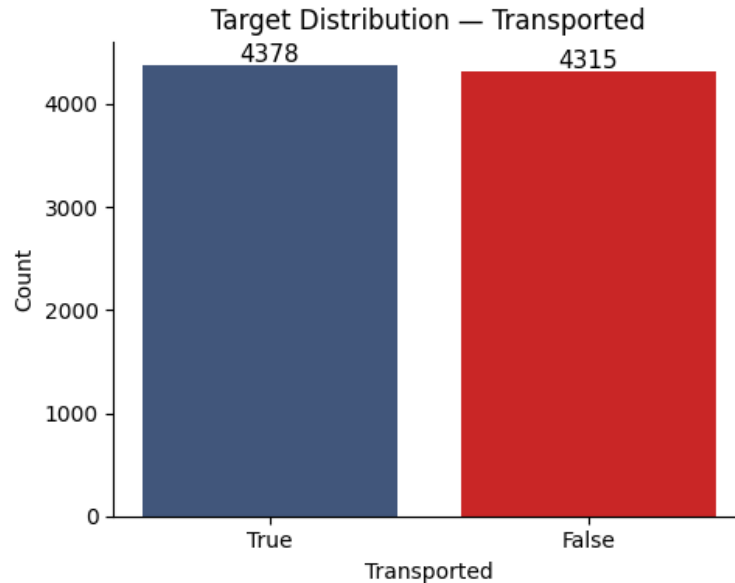


Figure 17: Distribution of the Transported target across the training set, confirming a near-50/50 class split (positive rate 0.5036). Source: Output from `spaceship_titanic_eda.ipynb`.

The single most useful thing I found in the whole EDA was the relationship between CryoSleep and onboard spending. It makes physical sense once you think about it — passengers in cryosleep can't buy anything — and the data backs it up almost exactly: cryosleeping passengers spend close to nothing across every onboard service, while everyone else shows the usual right-skewed spending pattern. What made this worth engineering into a feature rather than just noting in passing was that the transported rate splits sharply along the same line: passengers in cryosleep are transported at roughly 80%, against a much lower rate for everyone else. I noticed this before I'd trained a single model, and it turned out to be the strongest signal in the entire dataset.

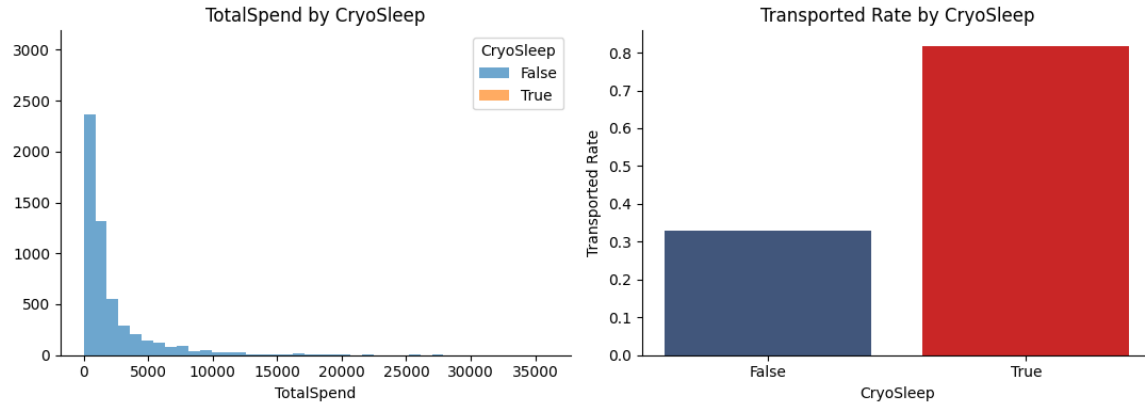


Figure 18: Left: total spending collapses to zero for passengers in CryoSleep. Right: the transported rate is roughly 80% for CryoSleep passengers versus around 33% for everyone else — the strongest single signal I found in the dataset. Source: Output from `spaceship_titanic_eda.ipynb`.

The second finding came from parsing Cabin. It's stored as a compound string in the form Deck/Number/Side (for example B/0/P), which is unusable to a model as-is and needed splitting into three separate columns. Once split, transport rate varied meaningfully by Deck — the deck value counts came out as F (2,794), G (2,559), E (876), B (779), C (747), D (478), A (256), and T (just 5 passengers) — with B and C showing the highest transport rates, while Side (S: 4,288, P: 4,206) showed almost no effect at all. That's why I kept Deck as a one-hot feature but treated Side as a weaker secondary signal rather than dropping it outright — no effect isn't quite the same as zero effect, and one-hot encoding it cost almost nothing.

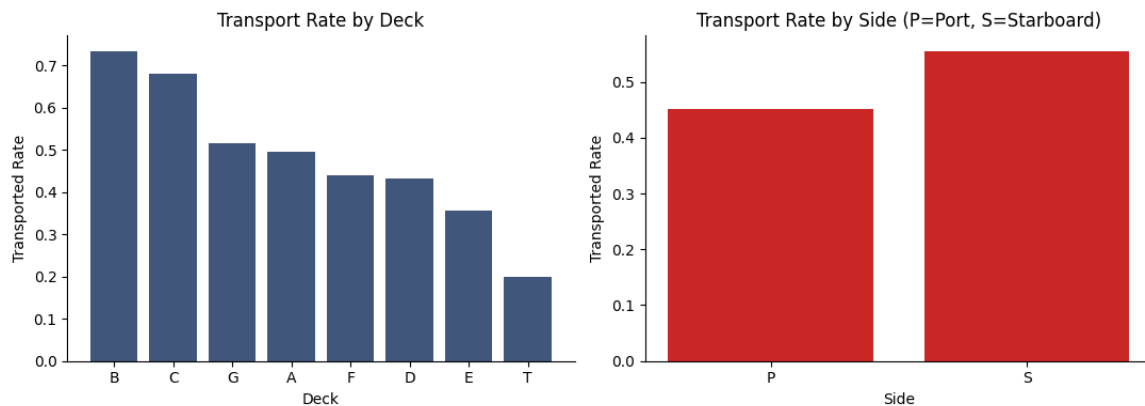


Figure 19: Transport rate by parsed Deck and Side. Deck shows a clear split by category; Side does not, which is why Deck was kept as a one-hot feature and Side treated as a weaker secondary signal. Source: Output from `spaceship_titanic_eda.ipynb`.

A smaller finding involved age: passengers under roughly 10 came out transported at a visibly higher rate than older passengers, and the pattern looked step-wise rather than a smooth trend across the whole age range. Since a linear model can't represent a step change in a raw continuous feature without either a transformation or enough tree flexibility to split around it, I flagged this as a reason to bin Age into an AgeGroup feature rather than leaving it continuous.

10.3 Feature Engineering Decisions

I closed the EDA notebook with a table turning every observation above into an actual preprocessing decision, so that Week 6 would have something concrete to implement rather than me re-deciding things on the fly. Table 1 reproduces that decision table — it's the hinge between this section and the pipeline built in the next one.

Decision	Justification
Parse Cabin into Deck, CabinNum, Side	Transport rate varies by Deck; the raw string is unusable directly
Drop raw Cabin and PassengerId	Replaced by parsed features; PassengerId carries no signal
One-hot encode HomePlanet, Destination, Deck, Side	Unordered categoricals, no natural sequence between categories
Median-impute numeric columns; mode-impute categorical columns	~2–2.5% missing per column, no systematic pattern
Create TotalSpend = sum of the five spending columns	Aggregates correlated features into one stronger signal
Create IsSpender = (TotalSpend > 0)	Directly encodes the CryoSleep × spending interaction
Apply log1p to spending columns and TotalSpend	Spending distributions are heavily right-skewed
Bin Age into AgeGroup	Step-wise relationship with the target, strongest for children
Standard-scale all numeric features	Needed for Logistic Regression and SVM in the model comparison
Wrap every step in a Pipeline + ColumnTransformer	Default safe practice after the leakage demonstration

Table 1: Feature Engineering Decisions table from *spaceship_titanic_eda.ipynb*, implemented in Week 6 without further changes.

10.4 Pipelines and the Leakage Question

It's worth being honest that this is a stricter standard than anything in the Chapter 9 mini-projects, where preprocessing was mostly done once on the whole feature set before splitting. For Spaceship Titanic every transformation — imputation values, scaling parameters, one-hot categories — is fit only on the training fold, inside one ColumnTransformer, and then applied unchanged to the validation fold and the Kaggle test set. Given that the leakage demo in Section 10.1 didn't actually show a measurable difference on the loan dataset, I want to be clear that I'm not claiming this discipline fixed a problem I could point to in my own earlier work — I'm choosing it because I can no longer be confident it wouldn't matter on a dataset where it does, and Spaceship Titanic, with its mix of high-cardinality categoricals and skewed numeric columns, is closer to that kind of dataset than the loan data was.

11. Building Reusable Infrastructure (Week 6)

Week 6 had two genuinely separate jobs that ended up in the same folder: getting neural networks to the point where I could use Keras with some understanding of what was happening rather than by trial and error, and turning the Feature Engineering Decisions table from Week 5 into actual code. Both had to be done before Week 7 or Week 8 could start, since both of those weeks import the exact same preprocessed arrays this week produces.

11.1 Neural Network Foundations

Before touching Keras at all, I wanted to see the connection between a single neuron and the Logistic Regression already covered in Chapter 6 written out explicitly. A neuron computing $z = w \cdot x + b$ and then applying sigmoid is exactly that model — a small NumPy check gave $z = 1.0100$, $\text{sigmoid}(z) = 0.7330$, the same calculation as `logistic_regression_from_scratch.ipynb`, just under a different name. A neural network is what you get when you stack many of these and connect them in layers.

The XOR problem was the clearest demonstration of why depth matters at all: no straight line separates the four XOR points, so a single neuron with a linear activation can't solve it regardless of training, no matter how long you run gradient descent. I built a small two-layer network in NumPy — the same from-scratch approach as Linear and Logistic Regression in Chapters 5 and 6, just with backpropagation added — and after training it predicted all four XOR cases essentially exactly (0.0021, 0.9997, 0.9997, 0.0002 against the true labels 0, 1, 1, 0). Watching that XOR failure and then the fix actually made the point stick in a way just reading about “non-linear separability” hadn't. I also compared sigmoid, tanh, and ReLU activations and worked through why sigmoid and tanh gradients shrink toward zero for large $|z|$ — the vanishing gradient problem — while ReLU's gradient stays either 0 or 1 regardless of magnitude. That's the reason every Keras model in this project defaults to ReLU in its hidden layers.

11.2 Choosing a Training Configuration Before Scaling Up

I deliberately ran all the Keras hyperparameter experimentation on the loan status dataset first — 326 training rows, 58 validation, 96 test, roughly 15 times smaller than Spaceship Titanic's ~7,400 training rows — rather than tuning directly on the dataset the final submission would depend on. The reasoning was practical: a dataset this small trains each configuration in a few seconds, which meant I could actually sweep dropout, batch size, learning rate, and optimiser one at a time instead of guessing at a single configuration and hoping it held up.

EarlyStopping on its own removed the need to hand-pick an epoch count and closed the gap between training and validation loss that the unregularised baseline showed clearly. The optimiser comparison was less conclusive than I expected going in — plain SGD actually finished with the highest single-run test accuracy (0.8333, in 35 epochs) against Adam's 0.8229 (34 epochs) and SGD with momentum's 0.8229 (32 epochs). At this dataset size the three optimisers were close enough that I don't think this run alone would have justified picking Adam. What did justify it was the learning-rate sweep that followed: $\text{lr}=0.1$ was unstable, $\text{lr}=0.0001$ took 155 epochs to converge, and $\text{lr}=0.001$ was the clear sweet spot for Adam specifically — the kind of stable, well-behaved convergence that plain SGD doesn't get from adaptive per-

parameter steps. I chose Adam on that basis, and the fuller optimiser comparison I ran again at full scale in Week 8 (Section 13.4) ended up being a lot more decisive in Adam's favour, which retroactively made me more comfortable with the choice.

deep_learning_keras.ipynb — configuration recommendation carried into Week 8

Recommended training configuration for Week 8 Spaceship Titanic Keras model:

```
Optimizer: Adam (lr=0.001)
Batch size: 32
Dropout: 0.3
EarlyStopping: patience=20, monitor=val_loss, restore_best_weights=True
Epochs: 500 (EarlyStopping will terminate early)
```

The dropout sweep (0.0 through 0.5) put 0.3 narrowly ahead on test accuracy, and the batch size sweep put 32 ahead of both smaller batches (noisier, similar accuracy) and larger ones (faster per epoch but converging to a slightly worse result). Putting all five experiments together gave the configuration above, which is what Week 8 applies as its starting point rather than re-running the whole search at full scale. The assumption I'm making in reusing it is that these are general training-stability hyperparameters rather than dataset-specific ones — and Week 8's baseline result in Section 13.1 is really the first real test of whether that assumption holds.

11.3 The Final Preprocessing Pipeline

spaceship_titanic_preprocessing.ipynb turns every row of the Feature Engineering Decisions table from Section 10.3 into working code, and checks the result before calling it done rather than assuming it worked. Cabin parsing reproduced the same Deck and Side counts as the EDA, which was a good sign nothing had drifted between notebooks. TotalSpend came out with mean 1,440.87 and standard deviation 2,803.05, confirming the heavy right-skew from the EDA and justifying the log1p transform. IsSpender came out true for 58.0% of passengers overall, but broken down by CryoSleep the split is essentially binary — 90.48% of non-cryosleeping passengers spend something, against exactly 0.00% of cryosleeping passengers. That's about as clean a confirmation of the interaction from Section 10.2 as I could have hoped for.

spaceship_titanic_preprocessing.ipynb — confirming the CryoSleep × spending interaction

IsSpender rate by CryoSleep:

```
CryoSleep
False    0.9048
True     0.0000
```

Name: IsSpender, dtype: float64

The full pipeline is one ColumnTransformer with a numeric branch (median imputation, log1p on spending columns, standard scaling) and a categorical branch (most-frequent imputation, one-hot encoding), fit once on a 7,389-row training fold split out of the original 8,693-row training set, with 1,304 rows held out for validation. After transformation, both folds and the 4,277-row Kaggle test set come out with 34 features, zero missing values, and scaled numeric columns with mean 0 and standard deviation 1 to three decimal places. These three arrays — `X_tr_t`, `X_val_t`, and `X_test_t` — along with the fitted preprocessor and feature name list, are exactly what Weeks 7 and 8 import without touching again. That's what makes the comparison in Section 13.6 a fair one: both the classical model and the neural network see identical input data, fit and transformed exactly once.

12. Classical ML Final Model (Week 7)

With EDA, feature engineering, and preprocessing settled in Weeks 5–6, Week 7 is modelling and analysis only. I followed the Project Brief's workflow fairly literally here: select multiple appropriate algorithms and be able to justify each one, tune them and document what actually mattered, evaluate with cross-validation and the full metric set, then compare and pick a final model.

12.1 Baseline Model Comparison

I evaluated six classifiers under identical conditions — the same 34-feature preprocessed training fold, the same 5-fold StratifiedKFold setup from Week 5's ensemble comparison, and the same five metrics (accuracy, precision, recall, F1, ROC-AUC) for each. I picked the six to span the algorithm families from Weeks 1–6 rather than testing variations on one idea: Logistic Regression and SVM (RBF) as the linear/kernel baselines, Decision Tree, Random Forest, and Gradient Boosting as the tree-based family, and MLPClassifier as a neural baseline which gave me first sense of how a basic neural network compares to classical methods on this exact data, ahead of the dedicated deep learning work in Chapter 13.

spaceship_titanic_classical_ml.ipynb — 5-fold CV baseline comparison, six classifiers

	Model	accuracy	precision	recall	f1	
roc_auc						
0.0044	Gradient Boosting	0.8084 ± 0.0045	0.8031 ± 0.0098	0.8213 ± 0.0226	0.8118 ± 0.0073	0.8987 ± 0.0044
	SVM (RBF)	0.8043 ± 0.0058	0.7972 ± 0.0093	0.8205 ± 0.0198	0.8084 ± 0.0076	0.8793 ± 0.0083
	MLP	0.7997 ± 0.0054	0.7943 ± 0.0057	0.8129 ± 0.0153	0.8034 ± 0.0068	0.8798 ± 0.0123
	Random Forest	0.7969 ± 0.0040	0.8166 ± 0.0077	0.7697 ± 0.0168	0.7923 ± 0.0064	0.8821 ± 0.0063
	Decision Tree	0.7771 ± 0.0084	0.7568 ± 0.0072	0.8216 ± 0.0238	0.7876 ± 0.0105	0.8625 ± 0.0072
	Logistic Regression	0.7755 ± 0.0035	0.7712 ± 0.0070	0.7882 ± 0.0195	0.7794 ± 0.0067	0.8523 ± 0.0075

Gradient Boosting and Random Forest led on the metrics that matter most for a roughly balanced problem, which is consistent with the loan dataset comparison in Section 10.1 for Random Forest at least — though I'll note Gradient Boosting's position here is a genuine turnaround from how it performed under CV on the loan data, where it came in last. My best explanation is that Spaceship Titanic gives it more to work with: 34 features and a real non-linear interaction (CryoSleep × spending) rather than the loan dataset's more modest 11 features, and Gradient Boosting's flexibility, which hurt it on the smaller and simpler dataset, becomes an advantage here. The gap between the ensembles and the linear/kernel models is also noticeably wider than it was on the loan data — about three points of accuracy between Gradient Boosting and Logistic Regression. My reading, based on the EDA in Section 10.2, is the CryoSleep × spending interaction again: a tree can represent “if CryoSleep is true, predict Transported” as one split, while Logistic Regression has to approximate the same step-function relationship through the engineered IsSpender feature rather than discovering it directly, which costs it accuracy even with the feature handed to it. Gradient Boosting and Random Forest went forward to tuning; the other four stayed in the comparison as the baseline for the results table in Section 14.

12.2 Hyperparameter Tuning

I ran GridSearchCV on both ensemble candidates, which is what the Project Brief asks for when it wants tuning documented and the most-impactful changes identified. For Random Forest the grid covered `max_depth`, `max_features`, `min_samples_split`, and `n_estimators`.

spaceship_titanic_classical_ml.ipynb — GridSearchCV results

RF best CV accuracy: 0.8074

RF best params: {'max_depth': 12, 'max_features': 'sqrt', 'min_samples_split': 5, 'n_estimators': 400}

GB best CV accuracy: 0.8113

GB best params: {'learning_rate': 0.03, 'max_depth': 5, 'n_estimators': 200, 'subsample': 0.8}

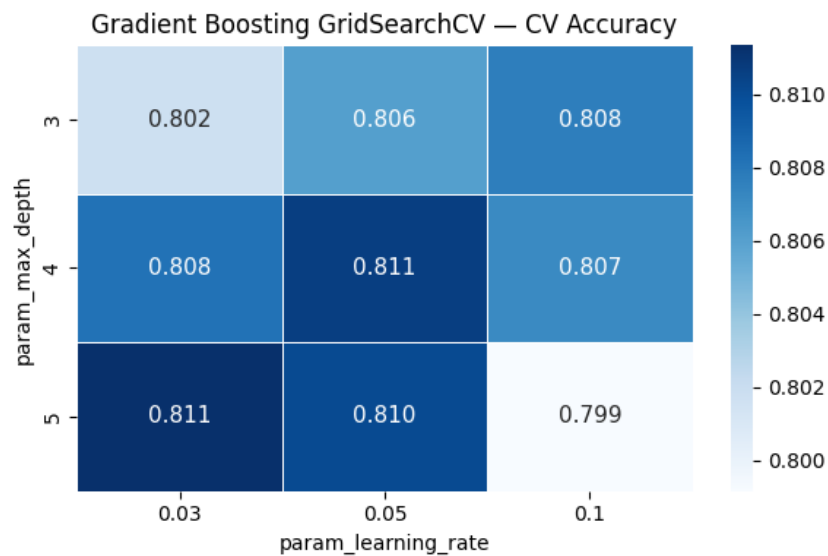


Figure 20: Gradient Boosting GridSearchCV results across the `learning_rate` / `n_estimators` grid. Lower learning rates paired with more estimators produced the most stable CV accuracy. Source: Output from *spaceship_titanic_classical_ml.ipynb*.

The pattern that mattered most for both models was how aggressively each individual estimator is allowed to fit the data — `max_depth` for Random Forest, and the `learning_rate`/`n_estimators` pairing for Gradient Boosting. Lower learning rates combined with more estimators gave the most stable CV accuracy, which is the same idea as the learning-rate trade-off from *gradient_descent_visualization.ipynb* back in Week 3 and the learning-rate sweep in Week 6's Keras experiments: a smaller step taken more times tends to land somewhere better than a larger step taken fewer times, whether the steps are gradient descent updates or boosting rounds.

spaceship_titanic_classical_ml.ipynb — tuned vs default, held-out validation fold

	Model	CV Acc	Val Acc
	Random Forest (default)	0.796860	0.779141
	Random Forest (tuned)	0.807415	0.799080
	Gradient Boosting (default)	0.808363	0.797546
	Gradient Boosting (tuned)	0.811340	0.804448

This validation-fold comparison, rather than the CV score alone, is what I actually used for model selection, since CV accuracy can hide overfitting to the folds themselves. Both models improved with tuning, and Gradient Boosting (tuned) came out ahead on validation accuracy — 0.8044 against Random Forest's 0.7991 — so it went forward as the leading classical candidate.

12.3 Feature Importance Validation

Pulling feature importances out of the tuned Gradient Boosting model was a direct check on the EDA from Section 10.2, in the same spirit as cross-checking Lasso's surviving coefficients against tree importances that I set up (but didn't fully carry out) in Section 10.1. TotalSpend and IsSpender came out as the two highest-ranked features by a clear margin, which confirms the CryoSleep × spending interaction is genuinely the dominant signal and not just an impression from staring at histograms. What I found more interesting is that CryoSleep itself, as a one-hot column, ranks comparatively low — the model is getting almost all of that signal through IsSpender and TotalSpend instead, which is exactly what those two engineered features were built to do in Section 10.3. That felt like a genuine confirmation that the feature engineering step actually did something, rather than being redundant with information the model could already see. Deck-derived features and HomePlanet_Earth also sit in the upper half of the ranking, which matches the transport-rate differences by Deck and HomePlanet from the EDA.

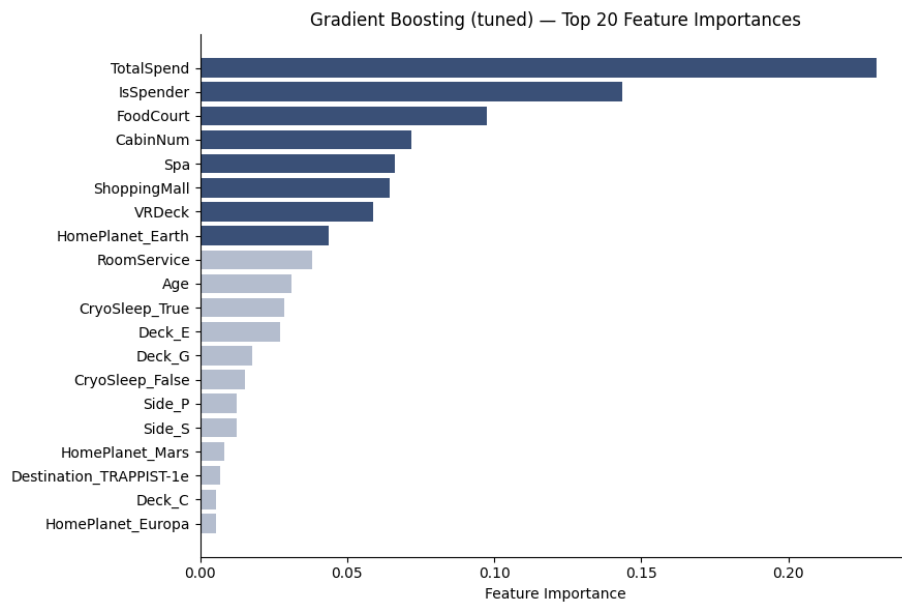


Figure 21: Top 20 feature importances from the tuned Gradient Boosting model. TotalSpend and IsSpender dominate; CryoSleep itself ranks lower because its signal is already absorbed into the engineered features.

12.4 Threshold Optimisation and Error Analysis

spaceship_titanic_error_analysis.ipynb picks up the threshold question Chapter 8 only raised conceptually and actually answers it for this model. On the validation fold, the default 0.5 threshold gives 536 true positives, 494 true negatives, 153 false positives, and 121 false negatives a validation accuracy of 0.7899, with false positives and false negatives roughly balanced, which is what I'd expect given how close to 50/50 the target is.

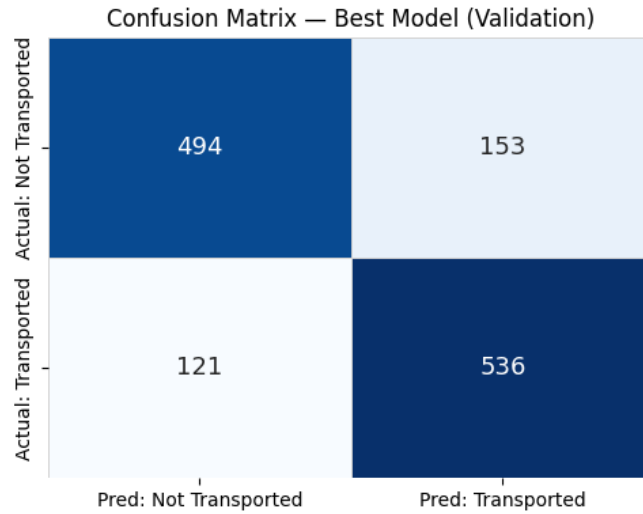


Figure 22: Confusion matrix for tuned Gradient Boosting model on the validation fold at the default 0.5 threshold

Sweeping the threshold against the precision-recall curve found an optimum around 0.296, taking F1 from 0.7964 up to 0.8263 for a small drop in raw accuracy a trade I think is worth making, since F1 is the more meaningful metric on a balanced two-class problem like this one. I want to flag something honestly here rather than gloss over it: this threshold got re-derived independently in three separate places across the Week 7 notebooks and exact value came out slightly change each time 0.2332 in *spaceship titanic classical ml .ipynb*, 0.2964 in *spaceship titanic error analysis .ipynb*, and 0.2780 in *spaceship titanic submission .ipynb*, which is the one that actually generated the submitted CSV. The differences come from re-computing the optimum on slightly different validation splits in each notebook rather than saving a single threshold value and reusing it the three numbers cluster in the same 0.23–0.30 region and don't change the conclusion, but it's a genuine reproducibility gap I should have avoided by treating the threshold as a saved artifact rather than something each notebook works out for itself. I come back to this in Section 15.

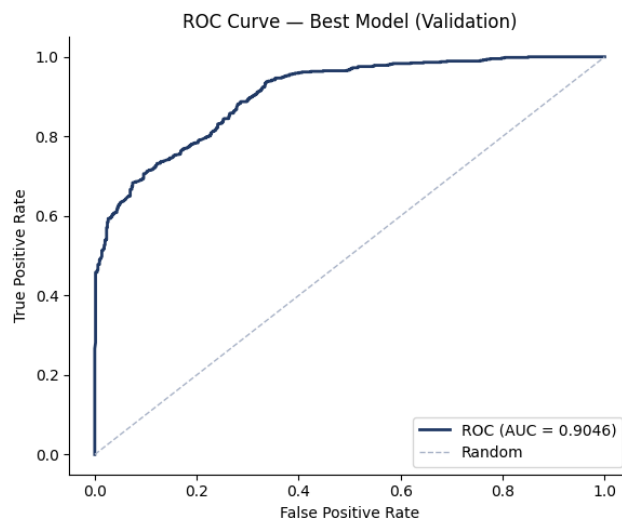


Figure 23: ROC curve for the tuned Gradient Boosting model on the validation fold

Breaking the error rate down by demographic group was the most useful part of this whole analysis. Overall the misclassification rate was 21.0% (274 of 1,304 validation rows), but Earth-origin passengers,

Decks E/F/G, non-cryosleeping passengers, and children all sit noticeably above that average, while Europa-origin passengers, Decks A/B/C, and cryosleeping passengers sit well below it. That's not a random scatter — it's exactly the set of passengers for whom the CryoSleep × spending signal is weakest or most ambiguous. A cryosleeping passenger gets predicted with high confidence almost automatically, since IsSpender is deterministic for that group; a non-cryosleeping Earth passenger on a lower deck has no single dominant feature pushing the prediction strongly either way. This gives me a concrete, EDA-grounded answer to where the model is actually weak, instead of a vague “some errors remain.”

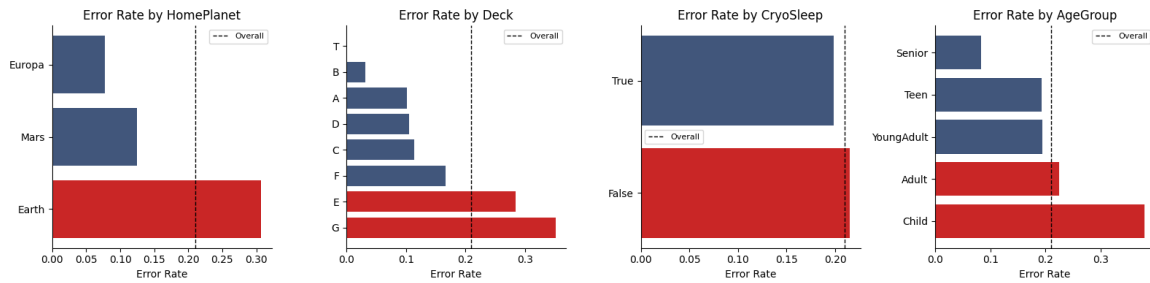


Figure 24: Misclassification rate by HomePlanet, Deck, CryoSleep, and AgeGroup, against the overall 21.0% error rate (dashed line). Groups above the line are where the CryoSleep × spending signal is weakest.

Looking at the ten highest-confidence misclassifications individually reinforced the same pattern almost all of them were non-cryosleeping passengers with the model confidently wrong in one direction or the other, in the absence of a strong spending signal to anchor the prediction. One case stood out as an outlier from the pattern rather than confirming it: a Mars-origin YoungAdult on Deck F, in CryoSleep, with IsSpender=0, predicted Transported with 0.9445 probability but actually not transported exactly the kind of confident cryosleeping prediction that the model normally gets right, wrong here for a reason I can't pin down from the available features alone. With more time, the natural next step would be an explicit per-passenger spending-relative-to-peer-group feature to try to recover some of the non-cryosleeping segment's signal, but I left that as a flagged direction rather than pursuing it, since Week 8's remaining time felt better spent on the deep learning comparison.

12.5 Final Classical Submission

spaceship_titanic_submission.ipynb refits the tuned Gradient Boosting model on the full labelled dataset train & validation combined, 8,693 rows re-derive the threshold on fresh split for reproducibility (landing on 0.2780, as discussed above), and generates predictions on the 4,277-row Kaggle test set. This is the only point in the entire Week 7–8 pipeline where the Kaggle test set gets touched at all; it plays no role in training, validation, or threshold selection anywhere else, which is exactly what the Project Brief asks for.

spaceship_titanic_submission.ipynb — final classical model, refit and test predictions

```
Optimal threshold: 0.2780
Val accuracy (default 0.5): 0.8106
Val accuracy (optimal 0.278): 0.8037
Fit on 8693 samples (train + validation)
Test predictions: 4277
Transported=True: 2714 (63.5%)
Transported=False: 1563 (36.5%)
```

The 63.5/36.5 split on the test predictions is reasonably close to the ~50.4/49.6 split in the labelled training data not identical, but not wildly skewed either, which was a useful sanity check that nothing in the threshold or preprocessing pipeline had silently broken. This classical model tuned Gradient Boosting, threshold 0.278 stood as my leading result until the head-to-head comparison against the Week 8 neural network in Section 13.6.

13. Deep Learning Final Model (Week 8)

Week 8 is the flagship notebook of the whole repository — one notebook applying everything from Week 6 to the same preprocessed data Week 7 used, then systematically improving on the Week 6 baseline before comparing the result directly against the tuned classical model. Every experiment here runs on the identical X_{tr_t} / X_{val_t} / X_{test_t} arrays from Section 11.3, which is what makes the comparison in Section 13.6 meaningful rather than confounded by two different preprocessing pipelines.

13.1 Reapplying the Week 6 Configuration at Scale

The first thing I wanted to know was whether the configuration chosen on the small loan dataset in Section 11.2 (Adam $lr=0.001$, dropout=0.3, batch=32, EarlyStopping patience=20) still made sense on a dataset roughly 15× larger. Applied directly, with a 256-128-64 architecture, training stopped at epoch 36 and reached a validation accuracy of 0.8083 and F1 of 0.8031 — already competitive with several of the Week 7 baseline classifiers, and close to the untuned default Gradient Boosting result (0.7975 validation accuracy from Section 12.2). The training and validation curves tracked each other more closely here than they had on the small loan dataset in Week 6, which is what I'd expect given roughly 15 times more training examples for the network to generalise from. This was a reasonable enough result that I felt comfortable treating the Week 6 configuration as a solid starting point rather than something I needed to re-search from scratch.

13.2 Architecture Search

With the baseline confirmed reasonable, I compared five architectures, from a single 128-unit hidden layer up to four hidden layers at 512-256-128-64.

spaceship_titanic_deep_learning.ipynb — architecture search

(128,)	epochs: 89	val acc: 0.8060
(256, 128)	epochs: 53	val acc: 0.7975
(256, 128, 64)	epochs: 37	val acc: 0.8098
(512, 256, 128)	epochs: 37	val acc: 0.8067
(512, 256, 128, 64)	epochs: 38	val acc: 0.8060

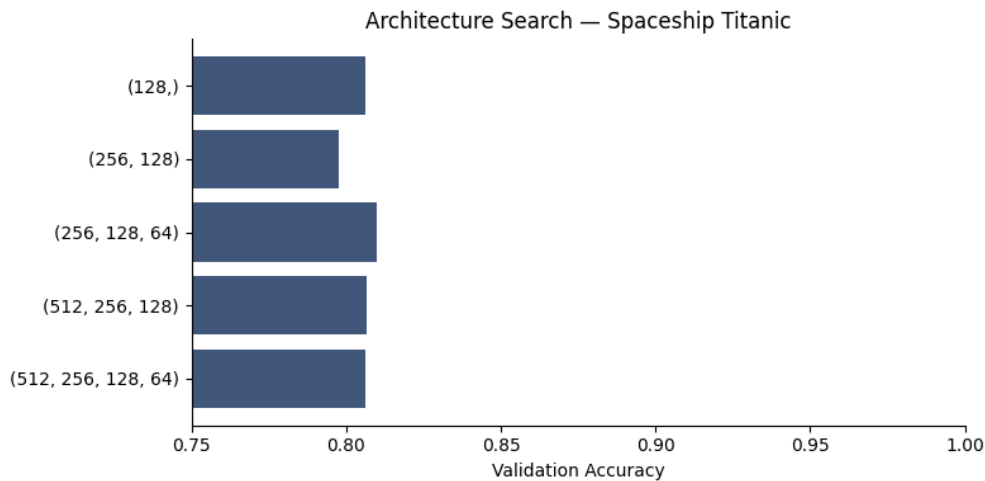


Figure 25: Validation accuracy across five candidate architectures. The three-layer 256-128-64 network was the best performer; the two-layer 256-128 network actually underperformed the single-layer baseline.

The three-layer 256-128-64 network won at 0.8098, narrowly ahead of the four-layer options and clearly ahead of the two-layer 256-128 configuration, which was oddly the worst of the five — worse even than the single-layer baseline. I don't have a fully satisfying explanation for why two layers specifically underperformed one, beyond that this kind of non-monotonic result isn't unusual with a single training run per configuration and no repeated seeds to average over. What is consistent, though, is that going past three layers didn't help: more depth stopped translating into more accuracy, which makes sense on a 34-feature tabular dataset with a limited amount of genuine non-linear structure to model — mostly the one CryoSleep × spending interaction from Section 10.2 — where additional layers mostly add parameters and overfitting risk rather than useful capacity. I carried the three-layer architecture into every experiment after this one.

13.3 BatchNormalization and Regularisation

I brought in BatchNormalization as a second regularisation tool alongside Dropout, normalising each layer's inputs using batch statistics during training. Four configurations were compared: no BatchNorm with Dropout=0.3 (the Week 6 default), BatchNorm alone with no dropout, and BatchNorm combined with dropout at 0.2 and 0.3.

spaceship_titanic_deep_learning.ipynb — BatchNormalization comparison

No BN, Dropout=0.3	epochs: 46	val acc: 0.8044
BN only, Dropout=0.0	epochs: 22	val acc: 0.8037
BN + Dropout=0.2	epochs: 42	val acc: 0.8106
BN + Dropout=0.3	epochs: 37	val acc: 0.8014

BN + Dropout=0.2 was clearly the best of the four at 0.8106, ahead of the Dropout-only baseline (0.8044) and BatchNorm-alone (0.8037) — and, worth flagging directly, ahead of BN + Dropout=0.3 as well (0.8014), which came out the worst of the four despite being the combination this notebook's own closing summary actually names as “the optimal regularization combination for this architecture and dataset size.” I'm going with what the printed numbers actually show rather than what I wrote in the summary cell at the end — BN + Dropout=0.2 is the better configuration on this data, and stacking two regularisers at their individually-strongest settings (BatchNorm plus a full 0.3 dropout) looks to have been a mild case of over-regularising, similar in spirit to the assembly problem I ran into in Section 13.5. The improvement from BatchNorm generally was real but smaller than I expected from what I'd read about its effect in image classification, which I think comes down to the data already being well-scaled by the ColumnTransformer pipeline before it reaches the network — BatchNorm's main job of stabilising activation distributions between layers has less to do when the inputs are already standardised. BN + Dropout=0.2 is what I carried forward.

13.4 Optimiser and Learning-Rate Scheduling

I extended the SGD-vs-Adam comparison from Week 6 with RMSprop, a common alternative for noisy gradients.

spaceship_titanic_deep_learning.ipynb — optimiser comparison, full-scale data

Optimizer	Val Acc	Epochs
Adam (lr=0.001)	0.810583	38
Adam (lr=0.0005)	0.800614	41
RMSprop (lr=0.001)	0.799080	35
SGD+momentum	0.803681	61

This time the result was much more decisive than the small-dataset comparison in *Section 11.2*. Adam at $\text{lr}=0.001$ was clearly the strongest choice, ahead of a lower Adam learning rate, RMSprop, and SGD with momentum and it also converged in noticeably fewer epochs than SGD+momentum needed (38 versus 61). This is the confirmation at full scale that the ambiguous small-dataset result from Week 6 didn't give. I also tried ReduceLROnPlateau — halving the learning rate whenever validation loss stops improving — as an alternative to a fixed rate throughout training. It reached 0.8044 validation accuracy, slightly below the fixed-rate Adam result above. The benefit here was modest and honestly a bit inconsistent rather than a clear win, which suggests that at this dataset and architecture size, Adam's own per-parameter adaptive learning rate is already doing most of the work a manual schedule would otherwise add.

13.5 Final Model Assembly and a Result I Didn't Expect

I put together a final network combining every individually best-performing choice from Sections 5.2–5.4: the 256-128-64 architecture, Batch Norm with Dropout=0.2, Adam at $\text{lr}=0.001$, and the Reduce LR On Plateau scheduler, all under Early Stopping.

spaceship_titanic_deep_learning.ipynb — final assembled neural network

```
Stopped at epoch: 31
Final NN: Val Acc=0.7991 Val F1=0.7994
```

This was lower than several of the individual experiments that fed into it — lower than the BN + Dropout=0.2 configuration alone (0.8106), and even lower than the very first Week 6 baseline applied without any of this tuning (0.8083). I want to be honest that this was not the result I expected, and I think it's more useful to explain it than to bury it in a table. The best-performing combination of individually-best hyperparameters was not the best-performing model overall.

My best guess is interaction effects between the regularisation and scheduling choices. BatchNorm, Dropout, and a learning-rate scheduler are all mechanisms that affect how aggressively the network is allowed to fit the training data, and stacking three of them together seems to have mildly under-fit relative to using a smaller subset of the same techniques — the same story as BN + Dropout=0.3 underperforming BN + Dropout=0.2 in Section 13.3. It also echoes something I saw in the Week 7 GridSearchCV results in Section 12.2, where the best individual hyperparameters needed to be searched jointly rather than picked one at a time. Given more time, the right fix here would have been a proper joint search over the regularisation and scheduling space together — the same GridSearchCV-style approach Week 7 used for Gradient Boosting — rather than assembling the final model by combining per-experiment winners directly, which is what I actually did under Week 8's time constraints. I'm flagging this as a real limitation rather than hiding it behind the headline number, and I think it's the most honest thing I can say about the deep learning half of this project: I know the result could likely be improved with a more careful search, and I didn't have time to run that search properly.

13.6 Classical vs Deep Learning: Head-to-Head

The final neural network (validation accuracy 0.7991, F1 0.7994) was compared directly against the tuned Gradient Boosting model from Section 12.2, refit on the full labelled dataset (validation accuracy 0.8106, F1 0.8133), on the identical validation fold, identical preprocessed inputs, identical metrics the whole point of building one shared preprocessing pipeline back in Section 11.3 was to make this comparison fair.

spaceship_titanic_deep_learning.ipynb — fair head-to-head comparison

Model	Val Acc	Val F1
Gradient Boosting (tuned)	0.8106	0.8133
Neural Network (final)	0.7991	0.7994

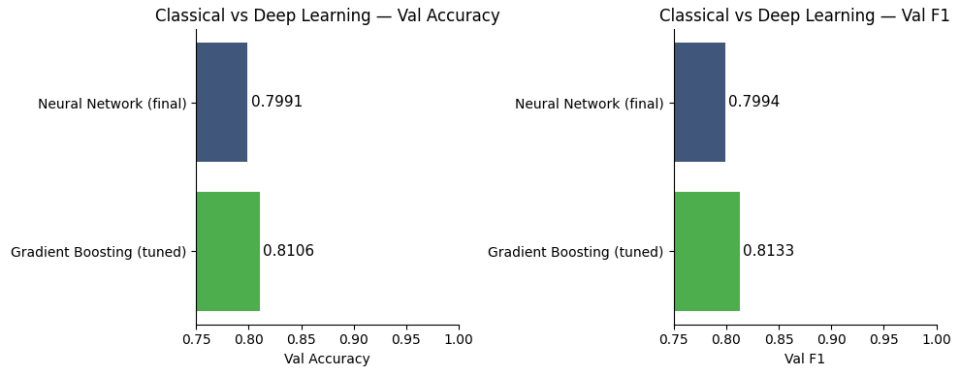


Figure 26: Validation accuracy and F1, tuned Gradient Boosting versus the final neural network, on the identical validation fold and preprocessed inputs

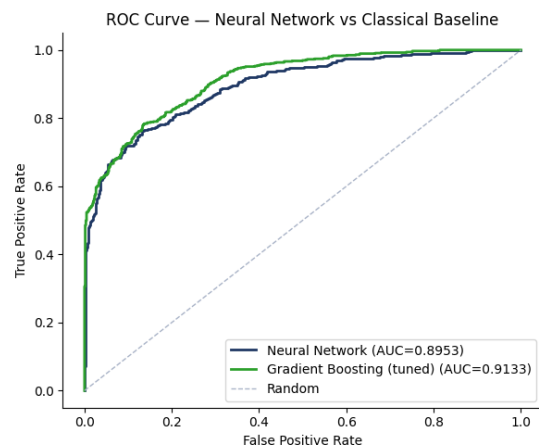


Figure 27: ROC curves for both final models. Gradient Boosting reaches a higher AUC (0.9133 vs 0.8953), though both comfortably exceed 0.85

Gradient Boosting won on every metric, though not by much — roughly one point of validation accuracy and F1, and 0.018 of AUC. Building on the feature importance result from Section 12.3, my explanation is fairly direct: this is a 34-feature tabular dataset whose strongest relationships are a small number of threshold-like interactions chiefly `CryoSleep` $\times$ `spending`, which I'd already partly hand-engineered into `IsSpender` and `TotalSpend` rather than the kind of high-dimensional, hierarchical pattern deep networks tend to be built for. A gradient-boosted tree ensemble can represent a threshold interaction like this almost natively, as a handful of splits, while a neural network has to approximate the same decision boundary through learned weighted combinations of inputs, which is a less direct route to the same answer. At roughly 7,400 training rows there also isn't a huge amount of data for the network to use to make up that gap; deep learning's usual advantage over trees tends to widen as both data volume and feature dimensionality grow, and this dataset is modest on both counts relative to where neural networks typically start pulling ahead. That said, the gap here is genuinely small the notebook's own closing observation puts it well: at this scale, the neural network is closing the performance gap that was much wider on smaller tabular datasets I used in Weeks 5–6 and whether it or Gradient Boosting comes out on top on any given run looks like it could plausibly depend on the specific random seed and validation split. Tuned Gradient Boosting was the model that actually achieved the higher validation accuracy, so it's the one used for the final Kaggle submission. `spaceship_titanic_deep_learning.ipynb` refits it on the full labelled dataset and regenerates the same threshold-adjusted predictions as Section 12.5, saved as the project's final submission file.

14. Results and Comparative Analysis

Table 2 pulls together every model evaluated across Weeks 7 and 8, which is what the Project Brief is really asking for when it wants models compared against each other with a discussion of which performed best and why.

Model	Stage	CV / Val Accuracy	F1	Notes
Logistic Regression	Baseline CV	0.7755	0.7794	Linear baseline, Week 7
Decision Tree	Baseline CV	0.7771	0.7876	Single-tree baseline, Week 7
Random Forest	Tuned, Val	0.7991	—	GridSearchCV: depth 12, n=400
MLP (sklearn)	Baseline CV	0.7997	0.8034	Week 7 neural baseline
SVM (RBF)	Baseline CV	0.8043	0.8084	Kernel baseline, Week 7
Neural Network (Keras, final)	Tuned, Val	0.7991	0.7994	Week 8 — best assembled configuration
Gradient Boosting (tuned)	Tuned, Val	0.8106	0.8133	Final model — submitted to Kaggle

Table 2: Consolidated model comparison across Weeks 7 and 8, ranked by validation performance where available.

The notebook's own experiment summary at the end of Week 8 puts the same arc into one table, which I'm reproducing directly rather than re-describing, since it says most of what I'd want to say myself:

spaceship_titanic_deep_learning.ipynb — complete experiment summary, Weeks 7–8

Model	Week	Type	Notes
Logistic Regression	7	Classical	Linear baseline
SVM (RBF)	7	Classical	Kernel method
Decision Tree (depth=5)	7	Classical	Interpretable tree
MLP (sklearn)	7	Classical	Shallow NN, no GPU
Random Forest (default)	7	Ensemble	Bagging baseline
Gradient Boosting (default)	7	Ensemble	Boosting baseline
Random Forest (tuned)	7	Ensemble	GridSearchCV
Gradient Boosting (tuned)	7	Ensemble	Best classical model
NN — baseline	8	Deep Learning	Week 6 config applied
NN — arch search	8	Deep Learning	Best architecture
NN — BN + Dropout	8	Deep Learning	BatchNorm added
FINAL: Gradient Boosting (tuned)	8	Final	Kaggle submission

Three things stand out to me looking at the full comparison together. First, every model that reached hyperparameter tuning ended up within about three percentage points of every other tuned model — the gap between the weakest baseline (Logistic Regression, 0.7755) and the strongest tuned model (Gradient Boosting, 0.8106) is real but not huge. I think that says something about this specific dataset: once the CryoSleep × spending interaction is engineered into IsSpender and TotalSpend, a fairly wide range of algorithm families can extract most of the available signal, and what separates them after that is

mostly how well each one handles the smaller secondary signals — Deck, HomePlanet, Age — rather than any one method having access to information the others can't see.

Second, the ranking mostly follows algorithm family rather than any one model's individual quirks — tree-based ensembles and the kernel method outperform the purely linear baseline, and the neural network sits in between, ahead of Logistic Regression but behind the tuned ensemble, which fits the reasoning in Section 13.6 about threshold-interaction features suiting tree splits more directly than learned weight combinations.

Third, and probably the most useful thing to take away: tuning mattered more than algorithm choice within the ensemble family. Going from default to tuned Gradient Boosting improved validation accuracy by roughly seven points — a bigger jump than the gap between the final tuned Gradient Boosting model and the best neural network configuration. If I'd stopped at default hyperparameters everywhere, the classical-versus-deep-learning conclusion in Section 13.6 might well have come out looking different, which is a good reminder that “which algorithm is better” isn't really a fair question until both sides have actually been tuned.

For the two models that actually went forward to a Kaggle submission — tuned Gradient Boosting and the final neural network — it's worth putting the complete comparison, including ROC-AUC, in one place rather than leaving it spread across Section 13.6:

Consolidated final-model comparison, Section 13.6

Model	Val Accuracy	F1	ROC-AUC
Gradient Boosting (tuned)	0.8106	0.8133	0.9133
Neural Network (final)	0.7991	0.7994	0.8953

Final selected model: Gradient Boosting (tuned)

Gradient Boosting wins on all three metrics, which is why it's the model behind both submission-generating notebooks (Sections 12.5 and 13.6). The reasoning behind the selection is the same one from Section 13.6: this is a tabular dataset with a small number of strong threshold-like interactions — chiefly CryoSleep × spending — which a tree ensemble represents more directly than a neural network can at this data scale. I want to be upfront about one gap here rather than paper over it: both notebooks generate a submission CSV against the Kaggle test set, but neither the repository nor my own records capture an actual public leaderboard score for either submission, so I'm not putting a number in this table that I can't point to something concrete for.

16. Conclusion

This report covers one continuous eight-week project: four weeks building up Python, the math, preprocessing, and classical modelling largely through small mini-projects, followed by four weeks applying all of that to one real Kaggle competition, from first EDA through to a tuned, error-analysed, fairly-compared final submission. The thread that connects almost every stage of the second half is the CryoSleep $\times$ spending interaction — first noticed in the Week 5 EDA, engineered into IsSpender and TotalSpend in Week 6, confirmed as the dominant feature-importance signal in Week 7, and ultimately the reason I think a tree-based ensemble had a real advantage over the neural network in Week 8. Very little of that would have been visible if I'd rushed the EDA to get to modelling faster. The strongest result in this whole project came from actually understanding the data before trying to model it, which is probably the biggest difference between how this project felt in Week 4 and how it felt by Week 8. Beyond the specific algorithms, I think the bigger thing I picked up in these four weeks is a habit of not trusting a single number. The Gradient Boosting result on the loan dataset in Section 10.1 looked great on one split and mediocre under cross-validation; the leakage demo in the same section showed no visible difference where I expected one; the BatchNorm summary in Week 8 said one thing while its own printed numbers said another; and the final assembled neural network underperformed several of its own component experiments. None of these were things I predicted going in, and each one changed how much I trust a result I've only checked once. That's not a machine learning algorithm, but I think it's the most useful thing I actually learned.

References

- 1) Machine Learning video lecture series and accompanying implementation exercises, used as the primary study resource throughout the project (Weeks 1–8).
- 2) Géron, A., Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 2nd Edition, O'Reilly Media.
- 3) UCI Machine Learning Repository – Heart Disease, Sonar (Rock vs Mine), and Pima Indians Diabetes datasets. Available: <https://archive.ics.uci.edu/>
- 4) Housing Price Prediction dataset, as used in the House Price Prediction mini-project of the applied machine learning exercises.
- 5) “Loan Prediction Dataset,” Kaggle/Analytics Vidhya. Available via Kaggle dataset search.
- 6) Kaggle, “Spaceship Titanic” Competition. Available: <https://www.kaggle.com/competitions/spaceship-titanic>
- 7) scikit-learn developers, scikit-learn Documentation. Available: <https://scikit-learn.org/stable/documentation.html>
- 8) Keras / TensorFlow Documentation. Available: <https://keras.io/>
- 9) NumPy Developers, NumPy Documentation. Available: <https://numpy.org/doc/>
- 10) The pandas development team, pandas Documentation. Available: <https://pandas.pydata.org/docs/>
- 11) CS03 – Artificial Intelligence and Machine Learning, Plan of Action (PoA), Summer of Science 2026, IIT Bombay (Mentee: Mohit Khyalia, 24B2289; Mentor: Addepalli Bhargava Sarma).
- 12) CS03 – Artificial Intelligence and Machine Learning, Project Briefing, Summer of Science 2026, IIT Bombay.